

# Region Convolution Neural Network

**R-CNN**      **Fast R-CNN**      **Faster R-CNN**

*Dr. Khaled Mostafa ElSayed*  
[kmelsayed@zewailcity.edu.eg](mailto:kmelsayed@zewailcity.edu.eg)

# Region Convolution Neural Network

R-CNN

Fast R-CNN

Faster R-CNN

# Region Based CNN

R-CNN

Fast R-CNN

Faster R-CNN

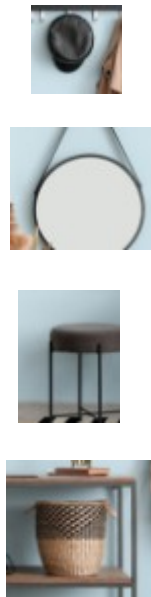
## Main Idea

Instead of Huge number of Sliding Window sub-images

**Propose some region ONLY for detector**



Region Proposals



Object Detector

- Refine Bounding Box
- Object Class

*Bounding Box  $x, y, W, H$   
for Proposed Regions*

*Up to 2000 proposed Regions*

# Region Proposal

# Region Based CNN

## R-CNN (Region proposal + CNN)

### There are many techniques for Region Proposals

Most common techniques:

- **Selective Search**  
(similarity of pixel colors, intensity, texture, ...)
- **Edge Boxes**
- Each region is extracted from the image then pass through CNN to calculate feature map.
- All regions are clipped to have the same size ( in order to have Same feature map size) to be passed to FC for classification and regression of Bounding Boxes.

# Region Proposal

## [1] Selective Search

### Selective Search



Color

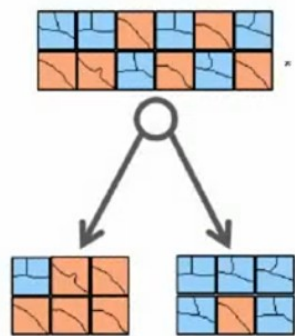
Texture

Circles and  
Curves

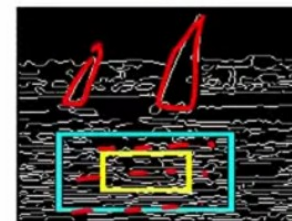
# Region Proposal

## [2] Edge Boxes

Edge Boxes ✓



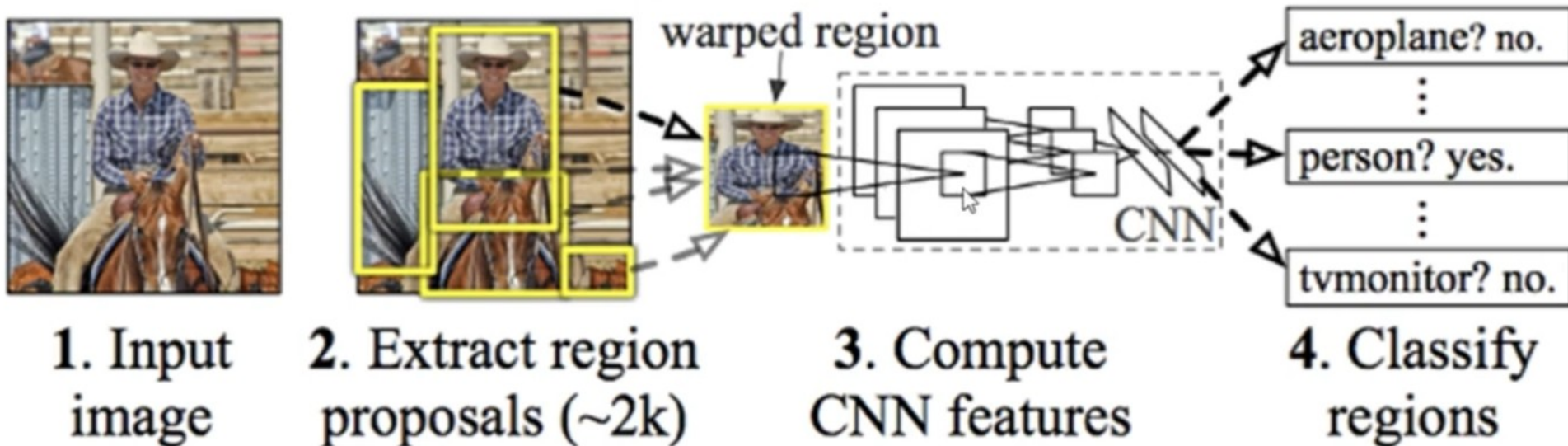
(c)



# Region Based CNN

R-CNN (Region proposal + CNN)

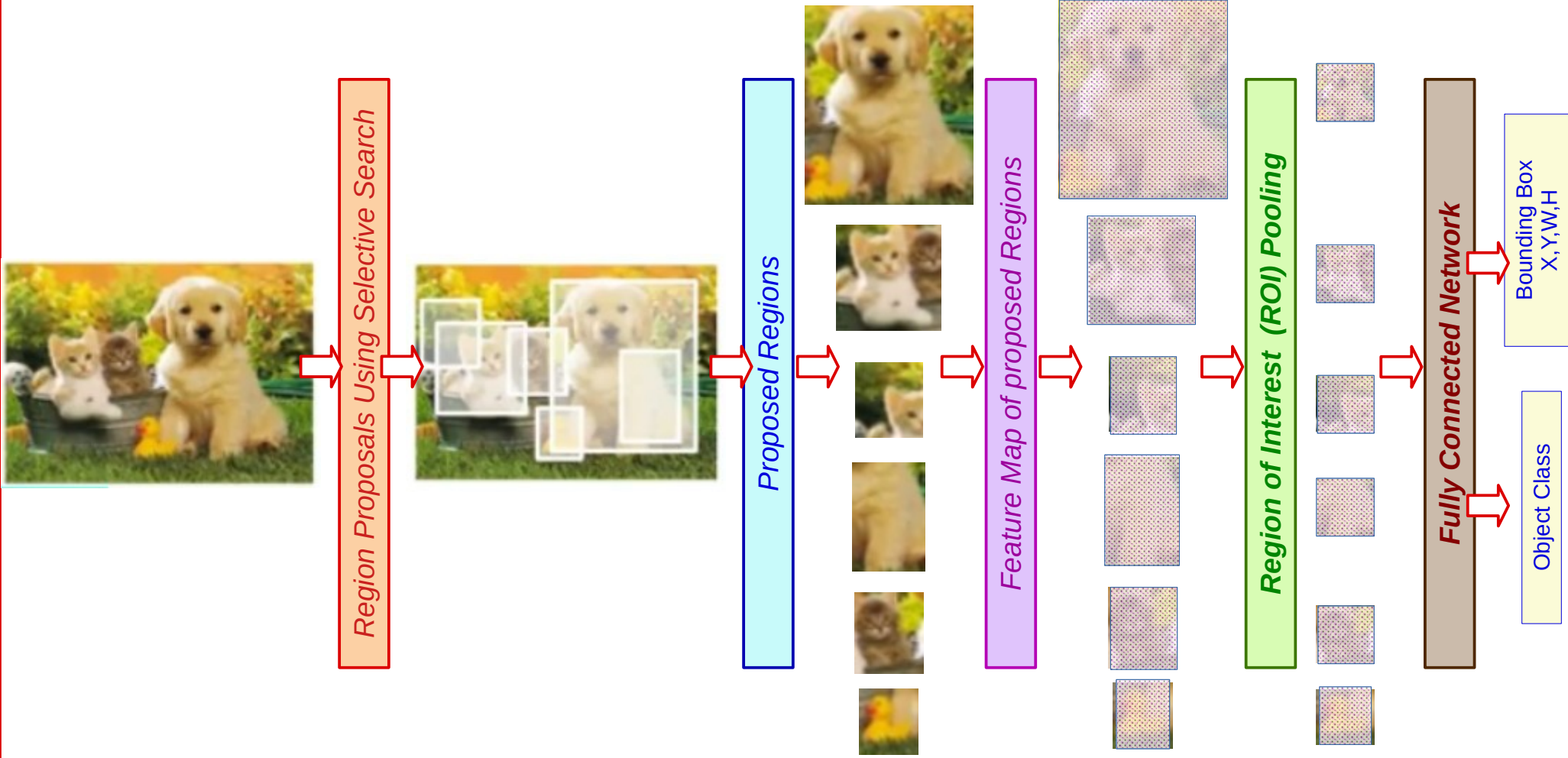
## R-CNN: *Regions with CNN features*





# RCNN

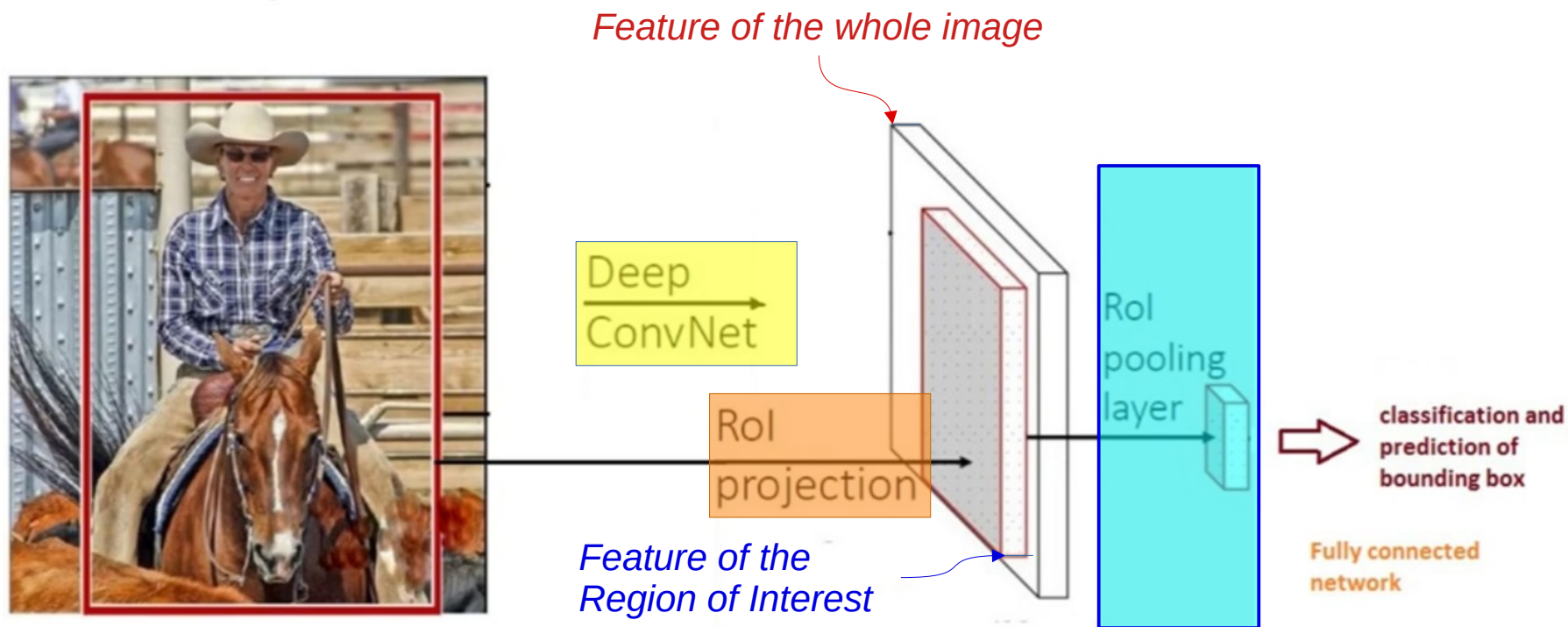
9



# Fast RCNN

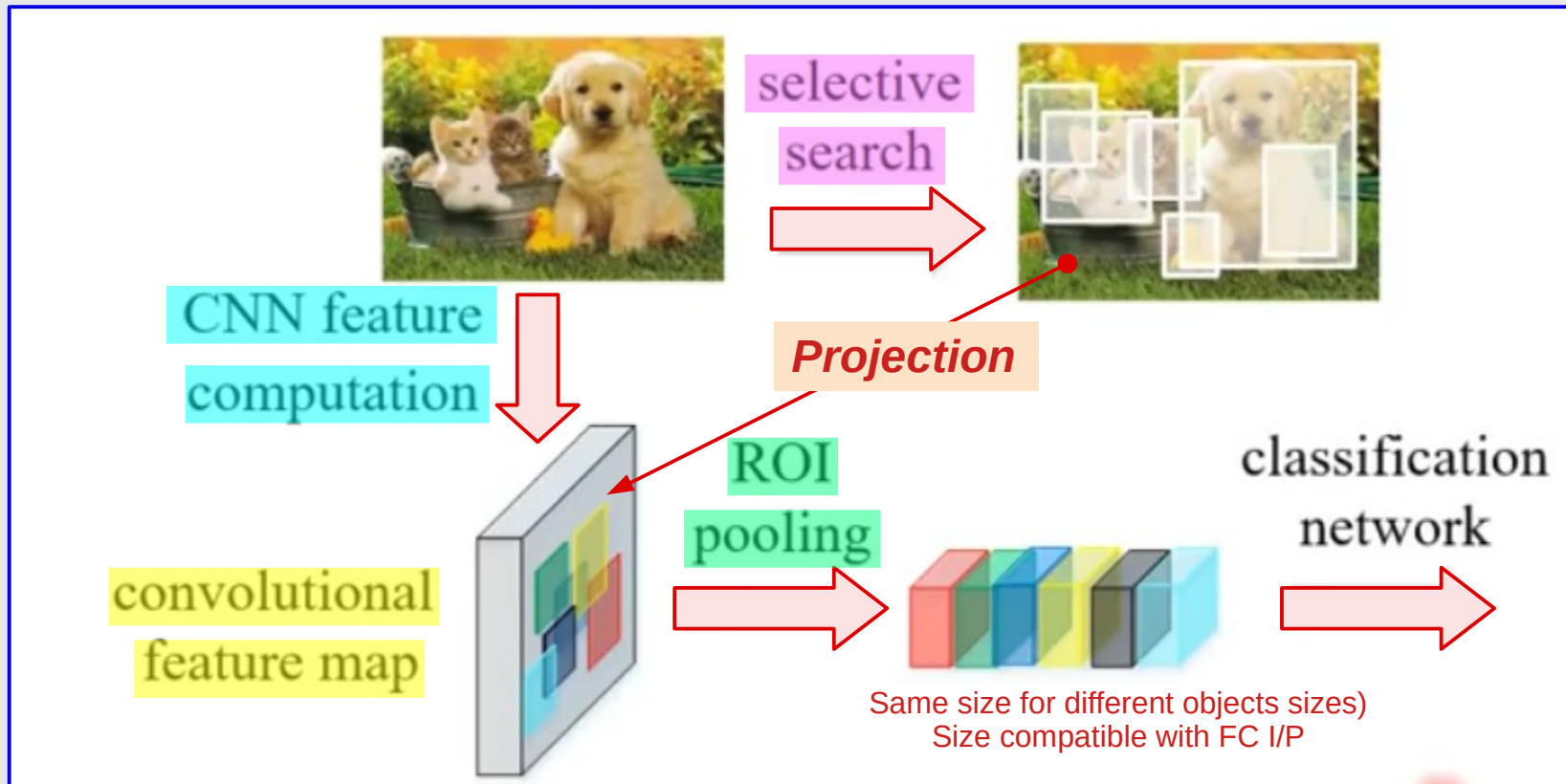
*(Extract Regions from Feature Map Domain)*

## Fast R-CNN



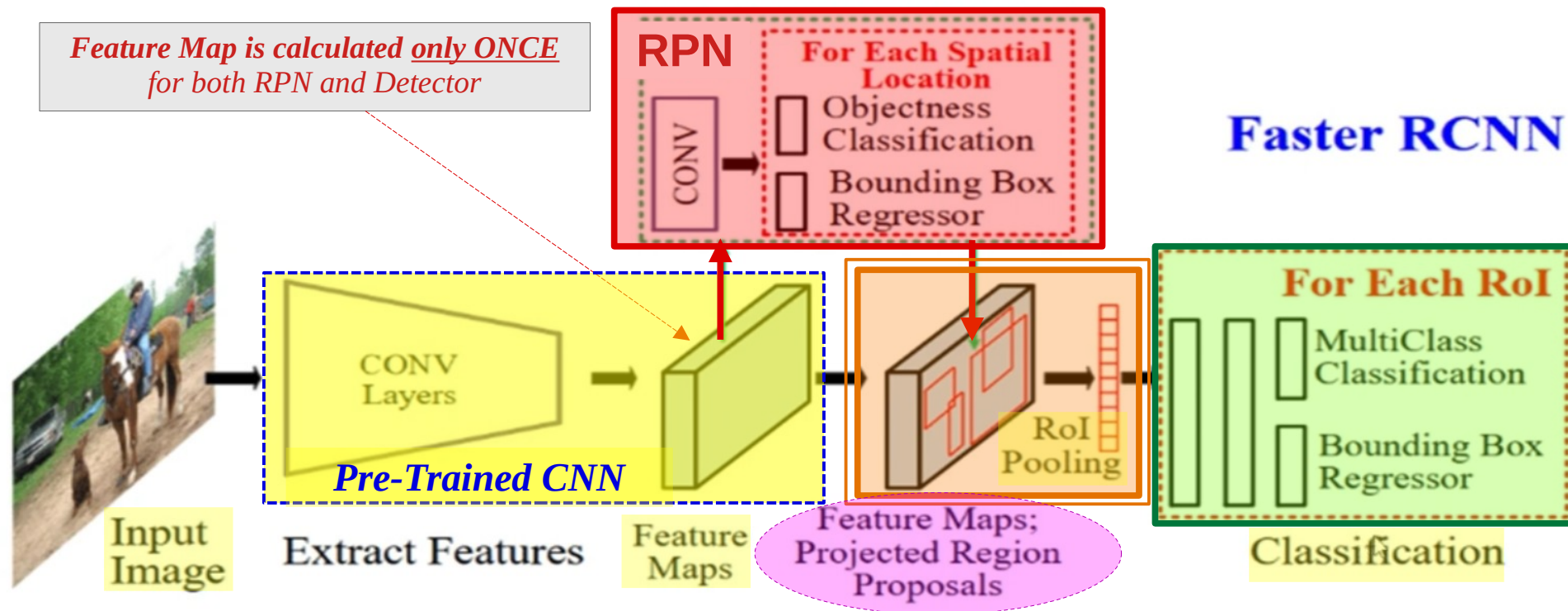
# Fast RCNN

*(Extract Regions from Feature Map Domain)*



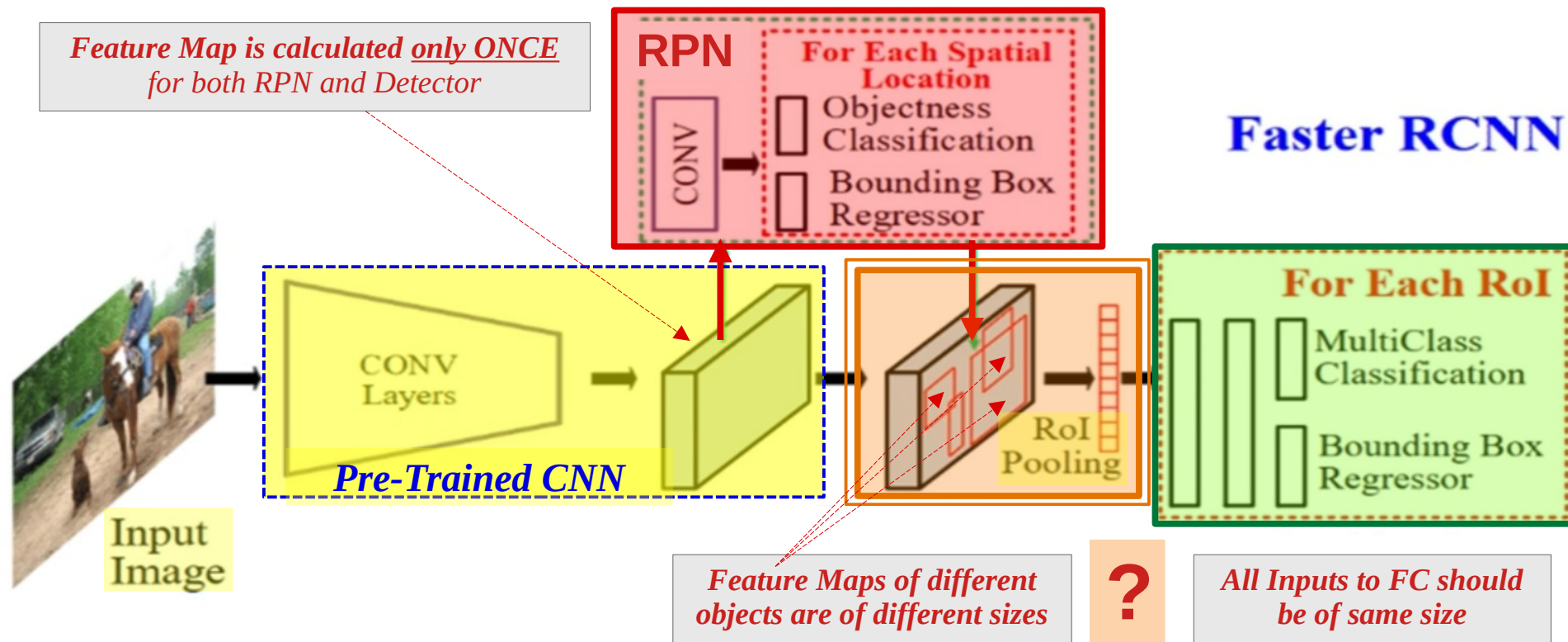
# Faster R-CNN

## FAST R-CNN + (Region proposal Network)



# Faster R-CNN

## FAST R-CNN + (Region proposal Network)



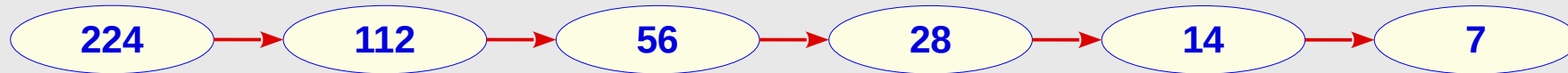
# ROI Pooling and ROI Align

***Input:***    *Different Feature map sizes*

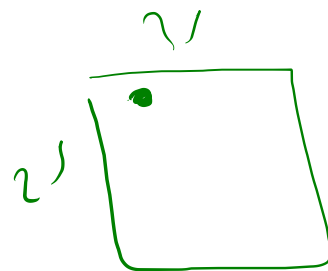
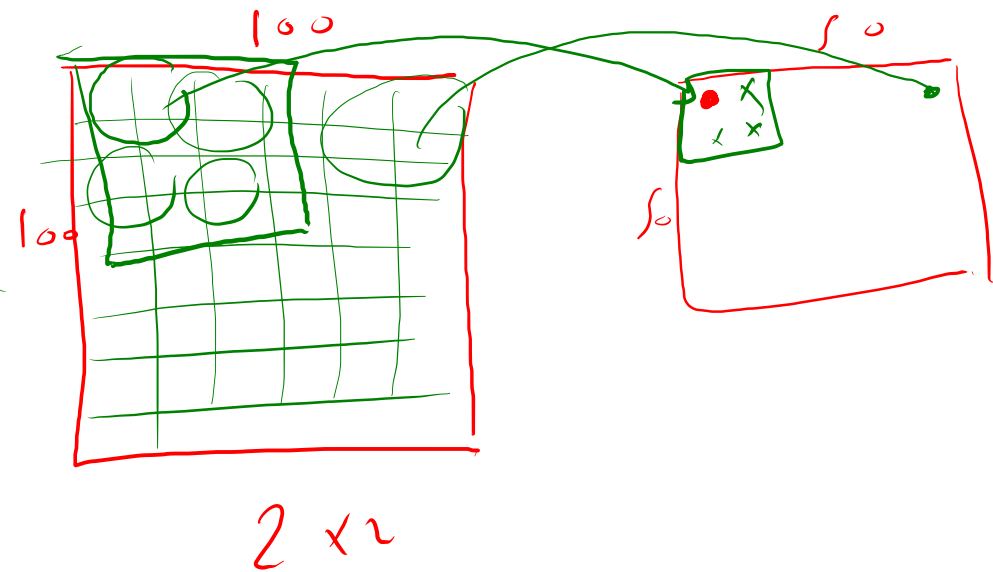
***Output:*** *Same Feature map size*

# **Sensitivity of Feature Map values**

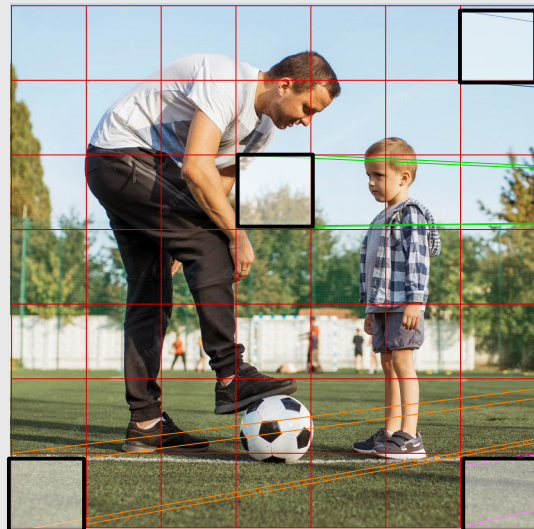




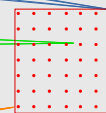




224 x 224

7x7x512  
Feature map

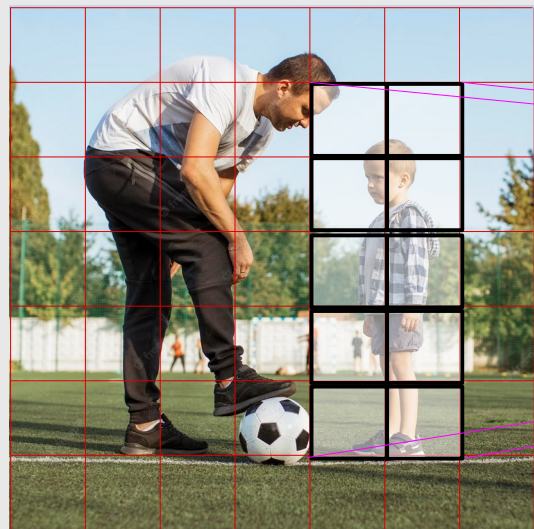
7



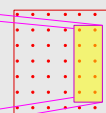
7

After 5 Pooling Layers  
224x224 is represented by 7x7 values  
(each value represents 32x32 pixels)

224 x 224

7x7x512  
Feature map

7

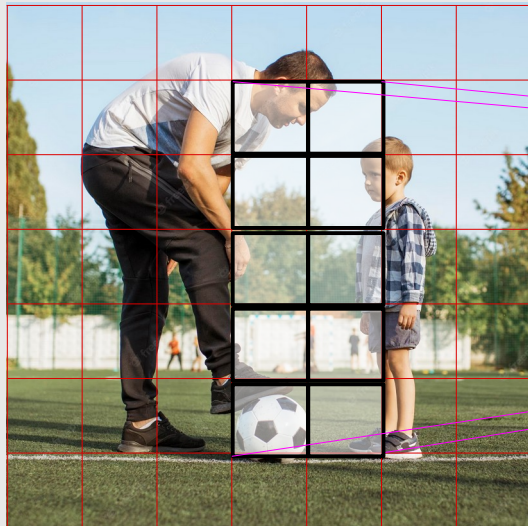


7

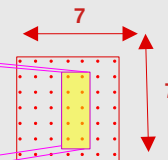
10 values in Feature Map are  
representing the Boy

# Effect of Shift and Crop of Feature Map

224 x 224



7x7x512  
Feature map

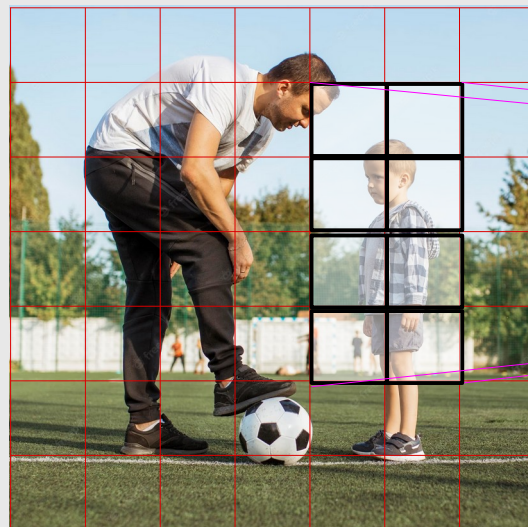


Effect of shift of values in feature map.

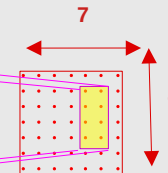
Missing Portion of the Boy

Adding portion of the father

[Error due to Quantization of Origin coordinates]



7x7x512  
Feature map

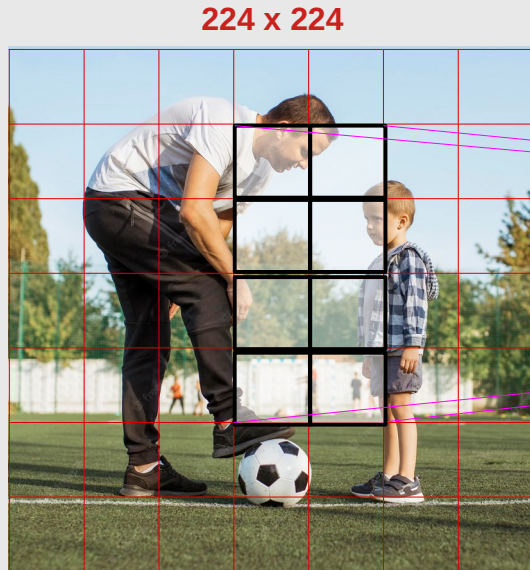


Effect of cropping some values in feature map.

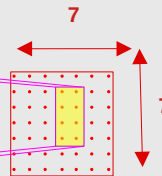
Missing Portion of the Boy

[Error due to Quantization of Width and Height]

## Effect of Both Shift and Crop of Feature Map



7x7x Chs  
Feature map



### Double Quantization Errors

[Error due to Quantization of Origin coordinates]

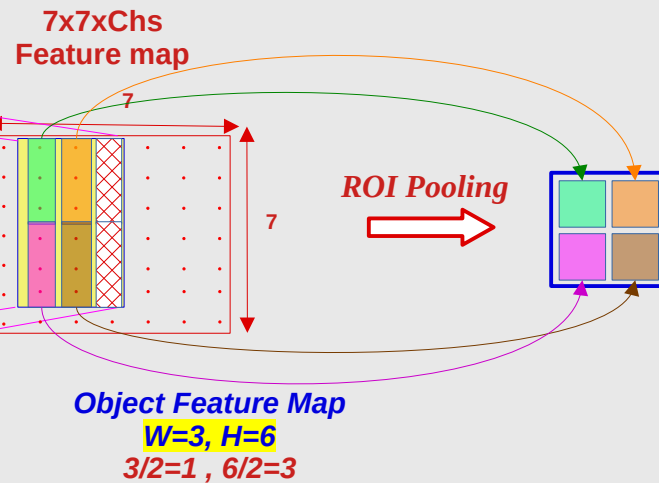
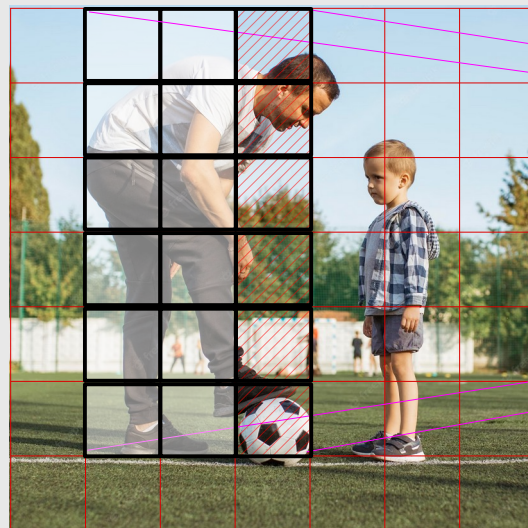
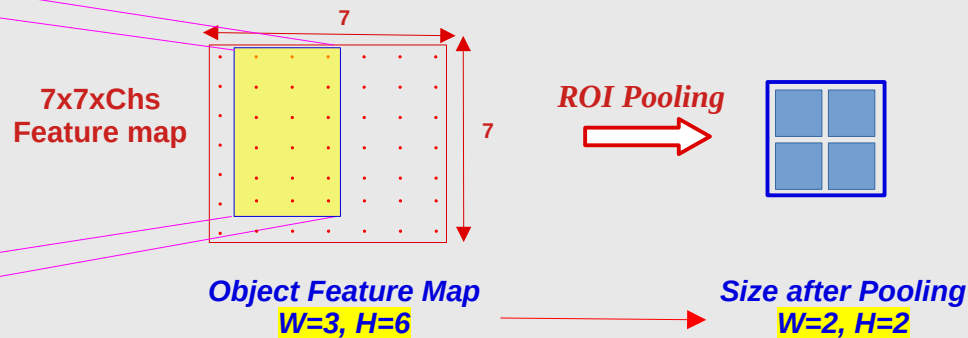
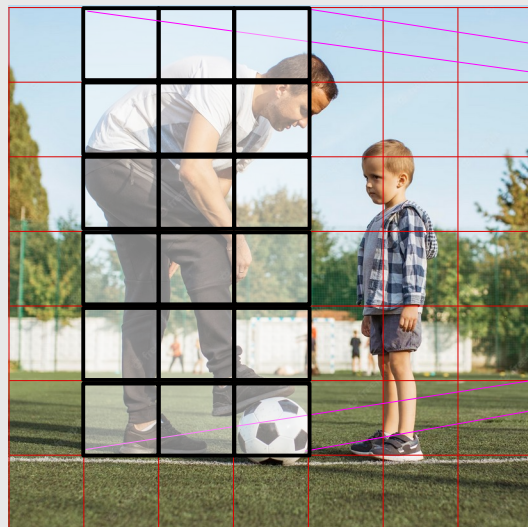
[Error due to Quantization of Width and Height]

*Quantization is a source of ERROR*

*But*

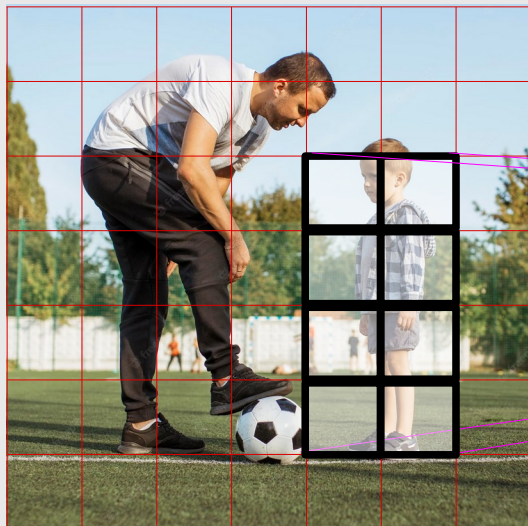
*Why do we need to Quantize ?*

# Region Of Interest (ROI) Pooling to 2 x 2 size

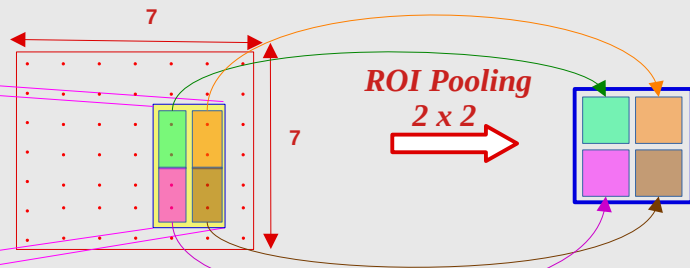


(Width Quantization error)

# Region Of Interest (ROI) Pooling to 2 x 2 size



7x7xChs  
Feature map



Object Feature Map

$W=2, H=4$

$2/2=1, 4/2=2$

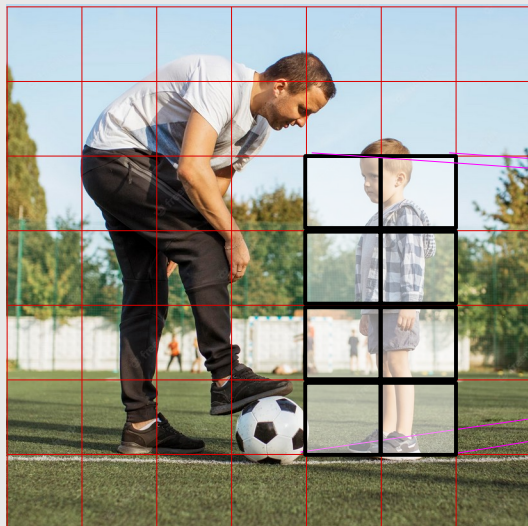
(NO Quantization error BUT !!!)

Size after Pooling

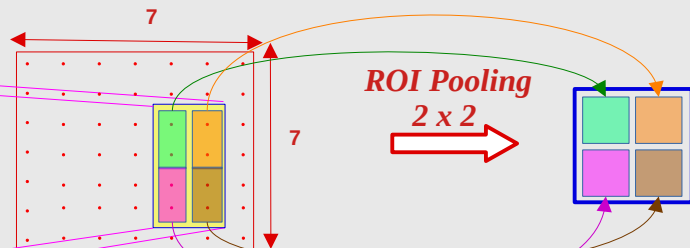
$W=2, H=2$



# Region Of Interest (ROI) Pooling to 2 x 2 size



7x7xChs  
Feature map



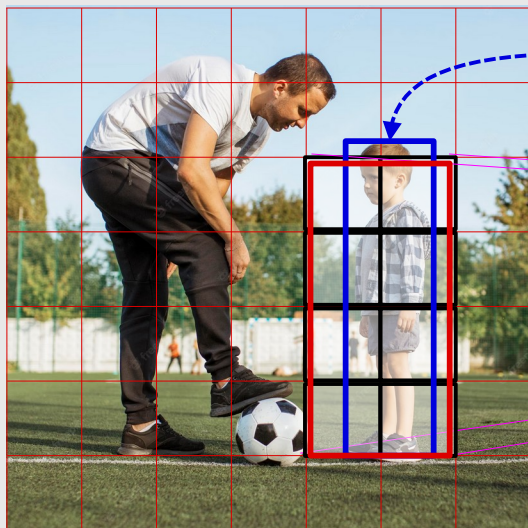
Object Feature Map  
 $W=2, H=4$

Size after Pooling  
 $W=2, H=3$

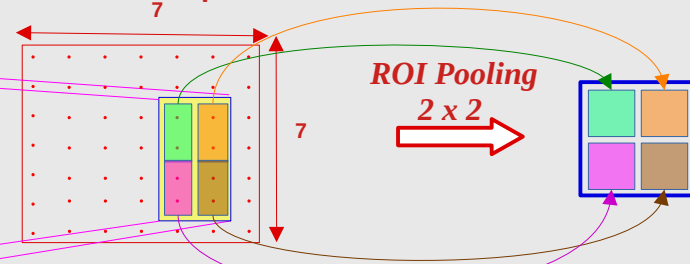
$$2/2=1, 4/2=3$$

(NO Quantization error BUT !!!)

Ground Truth  
Box



7x7xChs  
Feature map



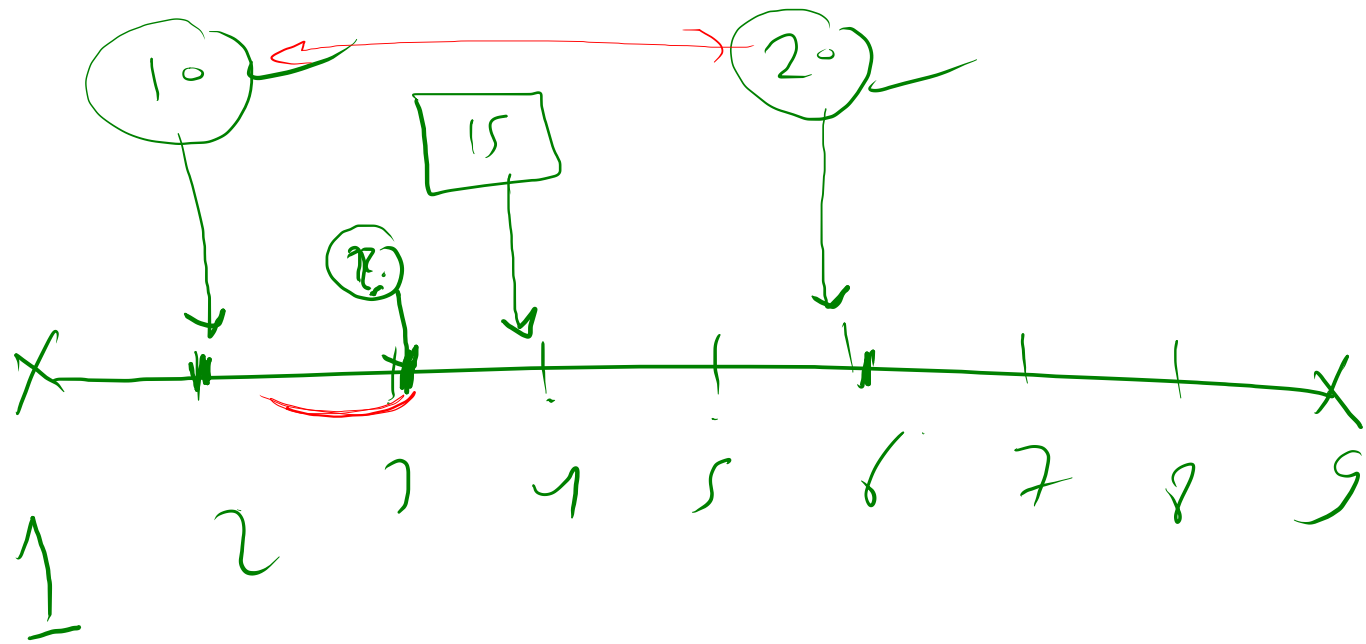
Box is NOT Properly aligned

Ground Truth Box (Blue)

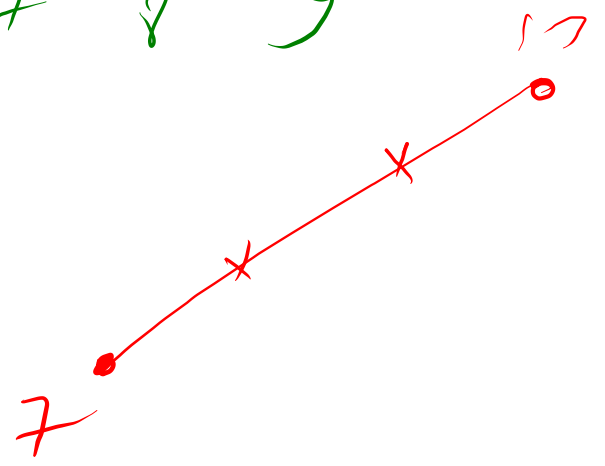
Box Corresponding to selected values in Feature Map (Red)

Remember: Each value in Feature Map corresponds to 16 x 16 cell





$10 + (20 - 10)$



# ROI Pooling: Problem Definition

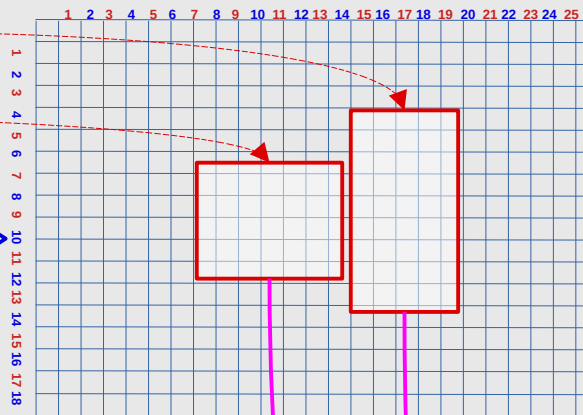
800 x 576 Image



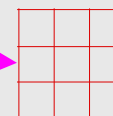
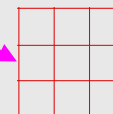
Region of Interest  
(ROI)

A Pre-Trained CNN Model  
(with 5 Pooling Layers)

25 x 18 Feature Map



ROI Pooling  
Output: 3x3  
(irrespective of input size)

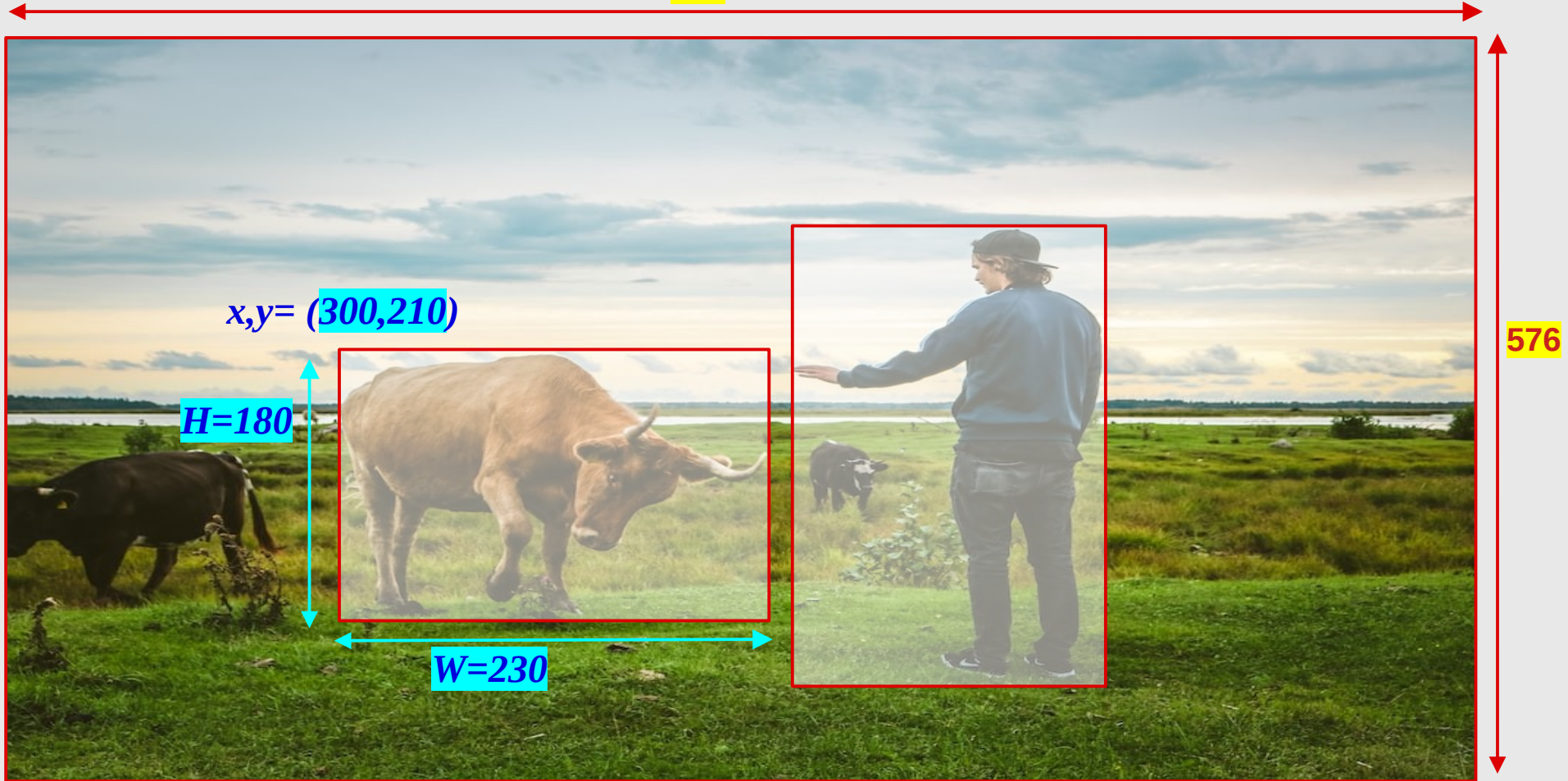


Fully Connected  
NN Classifier

# Annotated Image

24

800



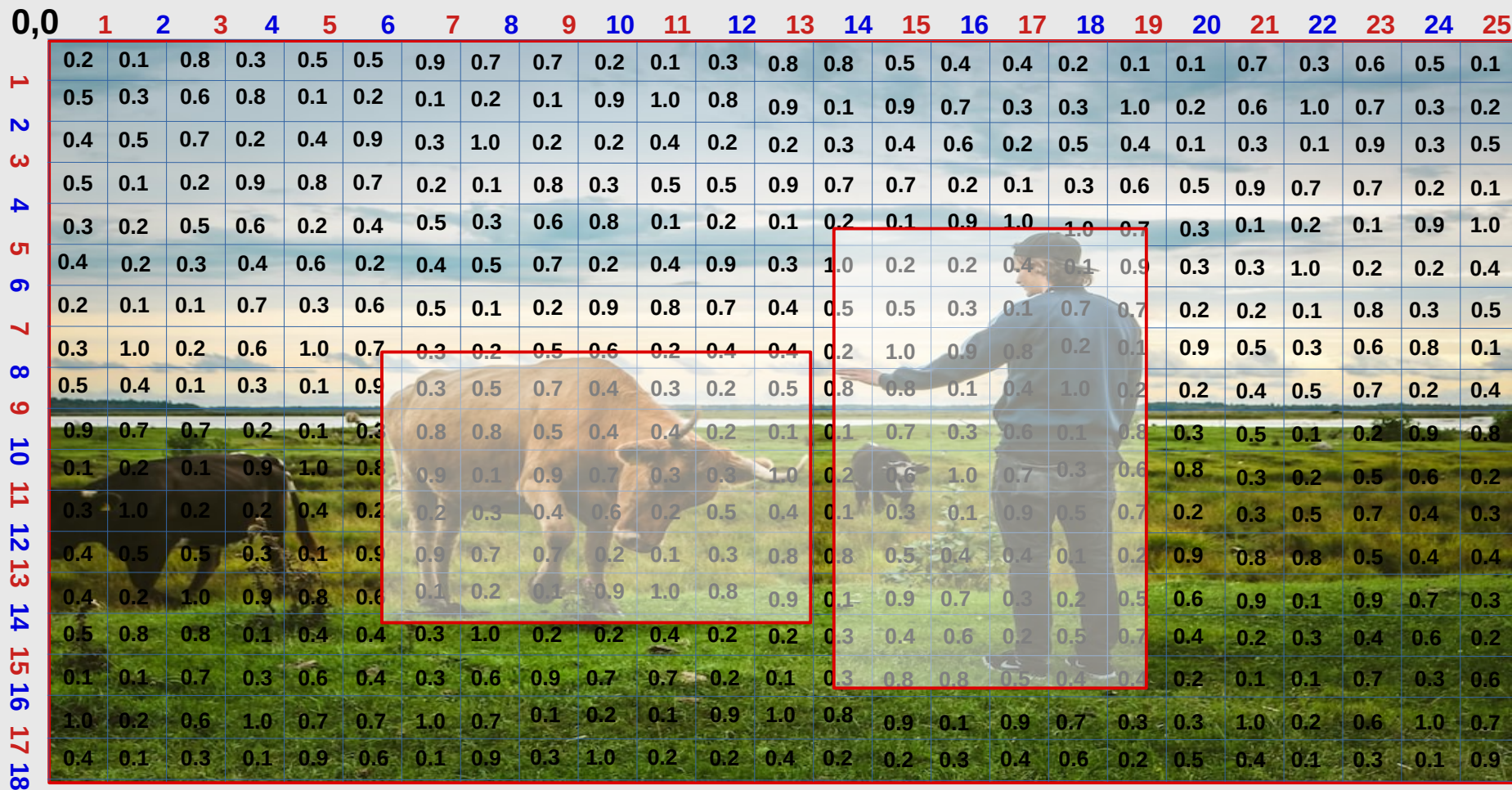
576

*Feature Map of the whole image*

Width = 800, Height=576, CNN with 5 Pooling Layers



Feature Map (800/32 , 576/32) = 25 x 18



Feature Map of the whole image

Width = 800, Height=576, CNN with 5 Pooling Layers

# Feature Map of selected object (Region of Interest [ROI])

| 0,0 | 1   | 2   | 3   | 4   | 5   | 6   | 7                            | 8   | 9             | 10  | 11                 | 12  | 13  | 14  | 15  | 16  | 17                                         | 18  | 19  | 20  | 21  | 22  | 23  | 24  | 25  |
|-----|-----|-----|-----|-----|-----|-----|------------------------------|-----|---------------|-----|--------------------|-----|-----|-----|-----|-----|--------------------------------------------|-----|-----|-----|-----|-----|-----|-----|-----|
| 1   | 0.2 | 0.1 | 0.8 | 0.3 | 0.5 | 0.5 | 0.9                          | 0.7 | 0.7           | 0.2 | 0.1                | 0.3 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4                                        | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.5 | 0.1 |
| 2   | 0.5 | 0.3 | 0.6 | 0.8 | 0.1 | 0.2 | 0.1                          | 0.2 | 0.1           | 0.9 | 1.0                | 0.8 | 0.9 | 0.1 | 0.9 | 0.7 | 0.3                                        | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.3 | 0.2 |
| 3   | 0.4 | 0.5 | 0.7 | 0.2 | 0.4 | 0.9 | 0.3                          | 1.0 | 0.2           | 0.2 | 0.4                | 0.2 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2                                        | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.3 | 0.5 |
| 4   | 0.5 | 0.1 | 0.2 | 0.9 | 0.8 | 0.7 | 300/32=9.375<br>210/32=6.563 |     |               |     | 0.5                | 0.5 | 0.9 | 0.7 | 0.7 | 0.2 | 0.1                                        | 0.3 | 0.6 | 0.5 | 0.9 | 0.7 | 0.7 | 0.2 | 0.1 |
| 5   | 0.3 | 0.2 | 0.5 | 0.6 | 0.2 | 0.4 | 0.5                          | 0.5 | 0.5           | 0.5 | 0.1                | 0.2 | 0.1 | 0.2 | 0.1 | 0.9 | 1.0                                        | 1.0 | 0.7 | 0.3 | 0.1 | 0.2 | 0.1 | 0.9 | 1.0 |
| 6   | 0.4 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2 | 0.4                          | 0.5 | (9.375,6.563) |     |                    | 0.9 | 0.3 | 1.0 | 0.2 | 0.2 | 0.4                                        | 0.1 | 0.9 | 0.3 | 0.3 | 1.0 | 0.2 | 0.2 | 0.4 |
| 7   | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.5                          | 0.1 | 0.2           | 0.9 | 0.8                | 0.7 | 0.4 | 0.5 | 0.5 | 0.3 | 0.1                                        | 0.7 | 0.7 | 0.2 | 0.2 | 0.1 | 0.8 | 0.3 | 0.5 |
| 8   | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.3                          | 0.2 | 0.5           | 0.6 | 0.2                | 0.4 | 0.4 | 0.2 | 1.0 | 0.9 | 0.8                                        | 0.2 | 0.1 | 0.9 | 0.5 | 0.3 | 0.6 | 0.8 | 0.1 |
| 9   | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.3                          | 0.5 | 0.7           | 0.4 | 0.3                | 0.2 | 0.5 | 0.8 | 0.8 | 0.1 | 0.4                                        | 1.0 | 0.2 | 0.2 | 0.4 | 0.5 | 0.7 | 0.2 | 0.4 |
| 10  | 0.9 | 0.7 | 0.7 | 0.2 | 0.1 | 0.3 | 0.8                          | 0.8 | 0.5           | 0.4 | 0.4                | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6                                        | 0.1 | 0.8 | 0.3 | 0.5 | 0.1 | 0.2 | 0.9 | 0.8 |
| 11  | 0.1 | 0.2 | 0.1 | 0.9 | 1.0 | 0.8 | 0.9                          | 0.1 | 0.9           | 0.7 | 0.3                | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7                                        | 0.3 | 0.6 | 0.8 | 0.3 | 0.2 | 0.5 | 0.6 | 0.2 |
| 12  | 0.3 | 1.0 | 0.2 | 0.2 | 0.4 | 0.2 | 0.2                          | 0.3 | 0.4           | 0.6 | 0.2                | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9                                        | 0.5 | 0.7 | 0.2 | 0.3 | 0.5 | 0.7 | 0.4 | 0.3 |
| 13  | 0.4 | 0.5 | 0.5 | 0.3 | 0.1 | 0.9 | 0.9                          | 0.7 | 0.7           | 0.2 | 0.1                | 0.3 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4                                        | 0.1 | 0.2 | 0.9 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4 |
| 14  | 0.4 | 0.2 | 1.0 | 0.9 | 0.8 | 0.6 | 0.1                          | 0.2 | 0.1           | 0.9 | 16.563-9.375=7.188 |     |     |     |     | 0.7 | 0.3                                        | 0.2 | 0.5 | 0.6 | 0.9 | 0.1 | 0.9 | 0.7 | 0.3 |
| 15  | 0.5 | 0.8 | 0.8 | 0.1 | 0.4 | 0.4 | 0.3                          | 1.0 | 0.2           | 0.2 | 0.4                | 0.2 | 0.2 | 0.3 | 0.4 | 0.6 | (300+230)/32=16.563<br>(210+180)/32=12.188 |     |     |     |     | 0.3 | 0.4 | 0.6 | 0.2 |
| 16  | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.4 | 0.3                          | 0.6 | 0.9           | 0.7 | 0.7                | 0.2 | 0.1 | 0.3 | 0.8 | 0.8 |                                            |     |     |     |     | 1   | 0.7 | 0.3 | 0.6 |
| 17  | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.7 | 1.0                          | 0.7 | 0.1           | 0.2 | 0.1                | 0.9 | 1.0 | 0.8 | 0.9 | 0.1 | 0.9                                        | 0.7 | 0.3 | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 |
| 18  | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.6 | 0.1                          | 0.9 | 0.3           | 1.0 | 0.2                | 0.2 | 0.4 | 0.2 | 0.2 | 0.3 | 0.4                                        | 0.6 | 0.2 | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 |

Feature Map Corresponding to Object at  $X,Y = (300,210)$   $W,H = (230,180)$

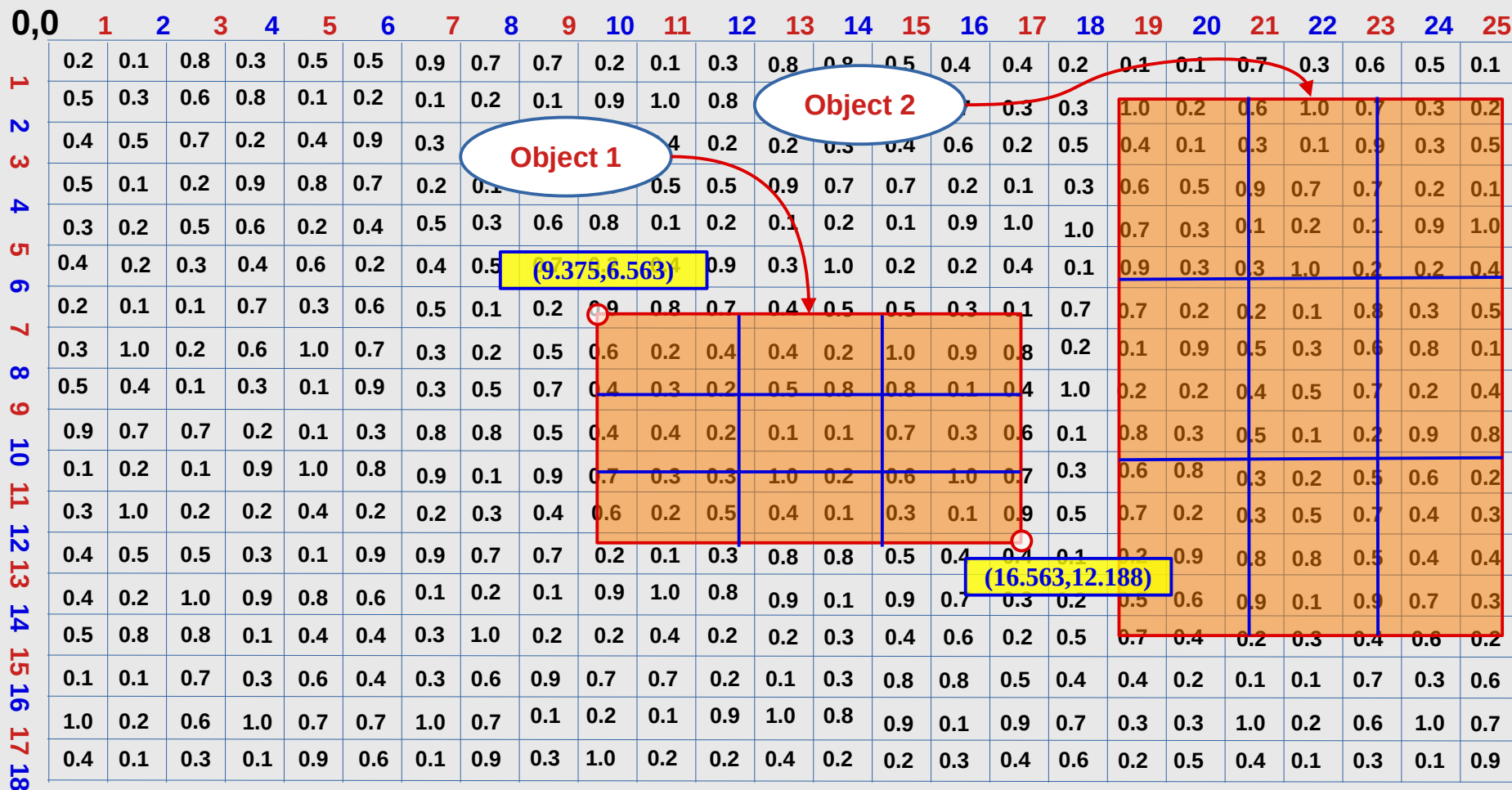
# ROI Pooling Output is 3x3

| 0,0 | 1   | 2   | 3   | 4   | 5   | 6   | 7   | 8   | 9             | 10  | 11  | 12  | 13  | 14  | 15  | 16  | 17  | 18  | 19  | 20  | 21  | 22  | 23  | 24  | 25  |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|---------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 1   | 0.2 | 0.1 | 0.8 | 0.3 | 0.5 | 0.5 | 0.9 | 0.7 | 0.7           | 0.2 | 0.1 | 0.3 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4 | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.5 | 0.1 |
| 2   | 0.5 | 0.3 | 0.6 | 0.8 | 0.1 | 0.2 | 0.1 | 0.2 | 0.1           | 0.9 | 1.0 | 0.8 | 0.9 | 0.1 | 0.9 | 0.7 | 0.3 | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.3 | 0.2 |
| 3   | 0.4 | 0.5 | 0.7 | 0.2 | 0.4 | 0.9 | 0.3 | 1.0 | 0.2           | 0.2 | 0.4 | 0.2 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2 | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.3 | 0.5 |
| 4   | 0.5 | 0.1 | 0.2 | 0.9 | 0.8 | 0.7 | 0.2 | 0.1 | 0.8           | 0.3 | 0.5 | 0.5 | 0.9 | 0.7 | 0.7 | 0.2 | 0.1 | 0.3 | 0.6 | 0.5 | 0.9 | 0.7 | 0.7 | 0.2 | 0.1 |
| 5   | 0.3 | 0.2 | 0.5 | 0.6 | 0.2 | 0.4 | 0.5 | 0.3 | 0.6           | 0.8 | 0.1 | 0.2 | 0.1 | 0.2 | 0.1 | 0.9 | 1.0 | 1.0 | 0.7 | 0.3 | 0.1 | 0.2 | 0.1 | 0.9 | 1.0 |
| 6   | 0.4 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2 | 0.4 | 0.5 | (9.375,6.563) |     |     | 0.9 | 0.3 | 1.0 | 0.2 | 0.2 | 0.4 | 0.1 | 0.9 | 0.3 | 0.3 | 1.0 | 0.2 | 0.2 | 0.4 |
| 7   | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.5 | 0.1 | 0.2           | 0.9 | 0.8 | 0.7 | 0.4 | 0.5 | 0.5 | 0.3 | 0.1 | 0.7 | 0.7 | 0.2 | 0.2 | 0.1 | 0.8 | 0.3 | 0.5 |
| 8   | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.3 | 0.2 | 0.5           | 0.6 | 0.2 | 0.4 | 0.4 | 0.2 | 1.0 | 0.9 | 0.8 | 0.2 | 0.1 | 0.9 | 0.5 | 0.3 | 0.6 | 0.8 | 0.1 |
| 9   | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.3 | 0.5 | 0.7           | 0.4 | 0.3 | 0.2 | 0.5 | 0.8 | 0.8 | 0.1 | 0.4 | 1.0 | 0.2 | 0.2 | 0.4 | 0.5 | 0.7 | 0.2 | 0.4 |
| 10  | 0.9 | 0.7 | 0.7 | 0.2 | 0.1 | 0.3 | 0.8 | 0.8 | 0.5           | 0.4 | 0.4 | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.1 | 0.8 | 0.3 | 0.5 | 0.1 | 0.2 | 0.9 | 0.8 |
| 11  | 0.1 | 0.2 | 0.1 | 0.9 | 1.0 | 0.8 | 0.9 | 0.1 | 0.9           | 0.7 | 0.3 | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.3 | 0.6 | 0.8 | 0.3 | 0.2 | 0.5 | 0.6 | 0.2 |
| 12  | 0.3 | 1.0 | 0.2 | 0.2 | 0.4 | 0.2 | 0.2 | 0.3 | 0.4           | 0.6 | 0.2 | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.5 | 0.7 | 0.2 | 0.3 | 0.5 | 0.7 | 0.4 | 0.3 |
| 13  | 0.4 | 0.5 | 0.5 | 0.3 | 0.1 | 0.9 | 0.9 | 0.7 | 0.7           | 0.2 | 0.1 | 0.3 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4 | 0.1 | 0.2 | 0.9 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4 |
| 14  | 0.4 | 0.2 | 1.0 | 0.9 | 0.8 | 0.6 | 0.1 | 0.2 | 0.1           | 0.9 | 1.0 | 0.8 | 0.9 | 0.1 | 0.9 | 0.7 | 0.3 | 0.2 | 0.5 | 0.6 | 0.9 | 0.1 | 0.9 | 0.7 | 0.3 |
| 15  | 0.5 | 0.8 | 0.8 | 0.1 | 0.4 | 0.4 | 0.3 | 1.0 | 0.2           | 0.2 | 0.4 | 0.2 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2 | 0.5 | 0.7 | 0.4 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2 |
| 16  | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 | 0.4 | 0.3 | 0.6 | 0.9           | 0.7 | 0.7 | 0.2 | 0.1 | 0.3 | 0.8 | 0.8 | 0.5 | 0.4 | 0.4 | 0.2 | 0.1 | 0.1 | 0.7 | 0.3 | 0.6 |
| 17  | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 | 0.7 | 1.0 | 0.7 | 0.1           | 0.2 | 0.1 | 0.9 | 1.0 | 0.8 | 0.9 | 0.1 | 0.9 | 0.7 | 0.3 | 0.3 | 1.0 | 0.2 | 0.6 | 1.0 | 0.7 |
| 18  | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 | 0.6 | 0.1 | 0.9 | 0.3           | 1.0 | 0.2 | 0.2 | 0.4 | 0.2 | 0.2 | 0.3 | 0.4 | 0.6 | 0.2 | 0.5 | 0.4 | 0.1 | 0.3 | 0.1 | 0.9 |

Feature Map of ROI (object) will be pooled to 3x3 (Align Pooling)

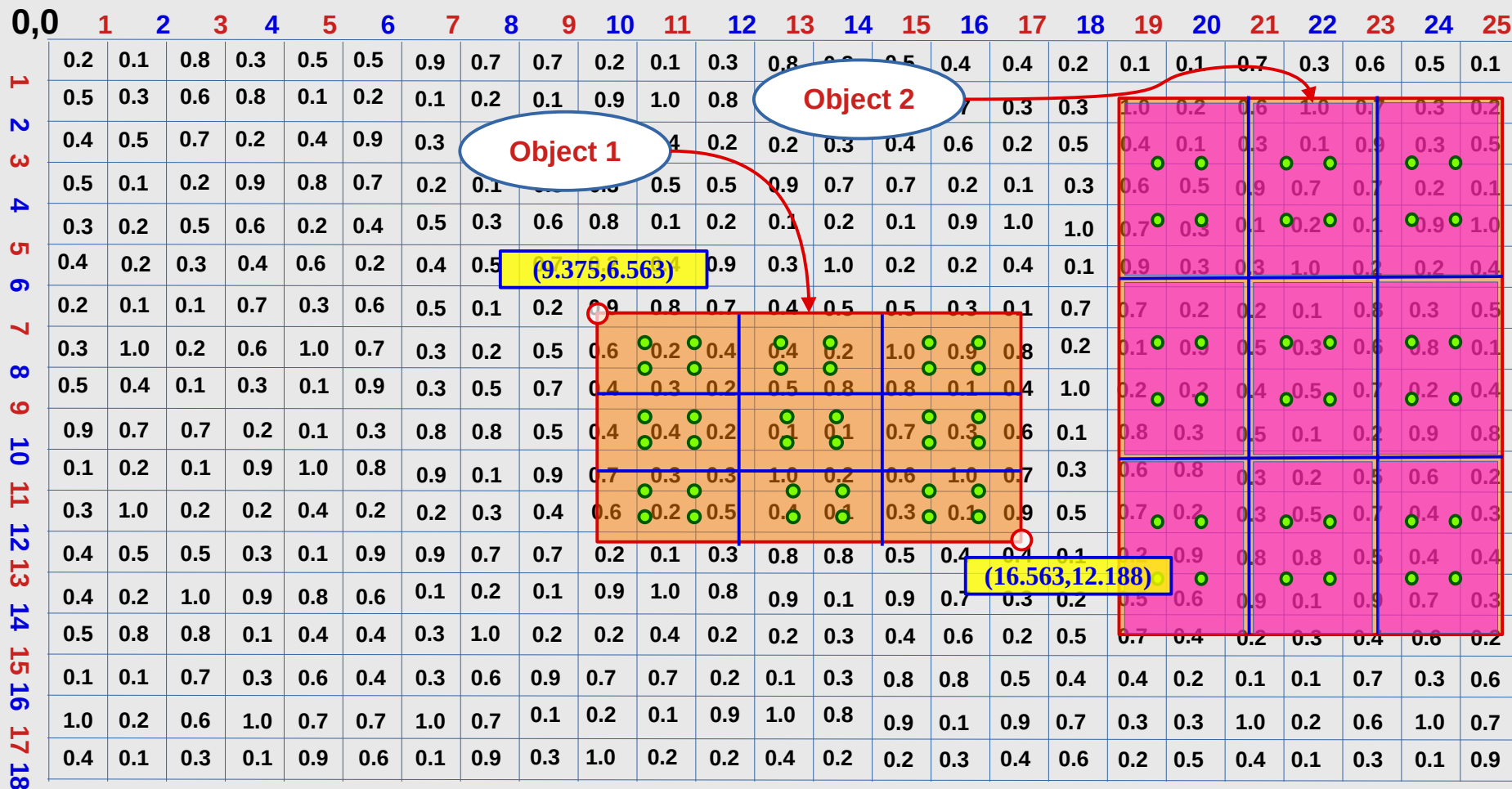


# ROI Pooling Output is 3x3 (Irrespective of Object Size / Aspect Ratio)



*Feature Map Corresponding to two Objects with Different Sizes (all will be Pooled to 3x3)*

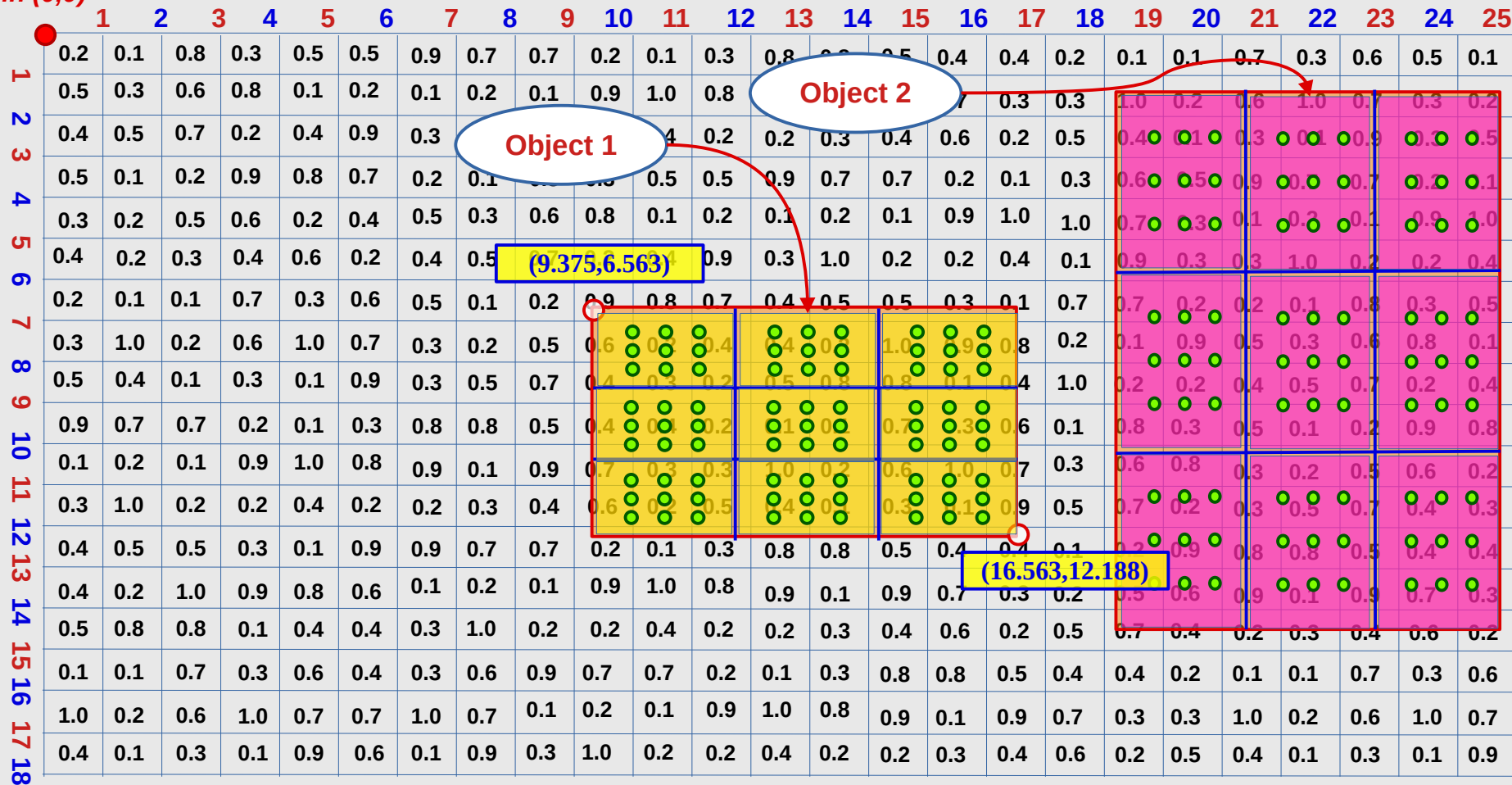
# Sampling Points “S<sub>x</sub>” represent values inside each cell (4 Points)



Select 2x2 Sampling Points (Calculate Corresponding Values then apply Max Pooling)

# Sampling Points “S<sub>x</sub>” represent values inside each cell (9 Points)

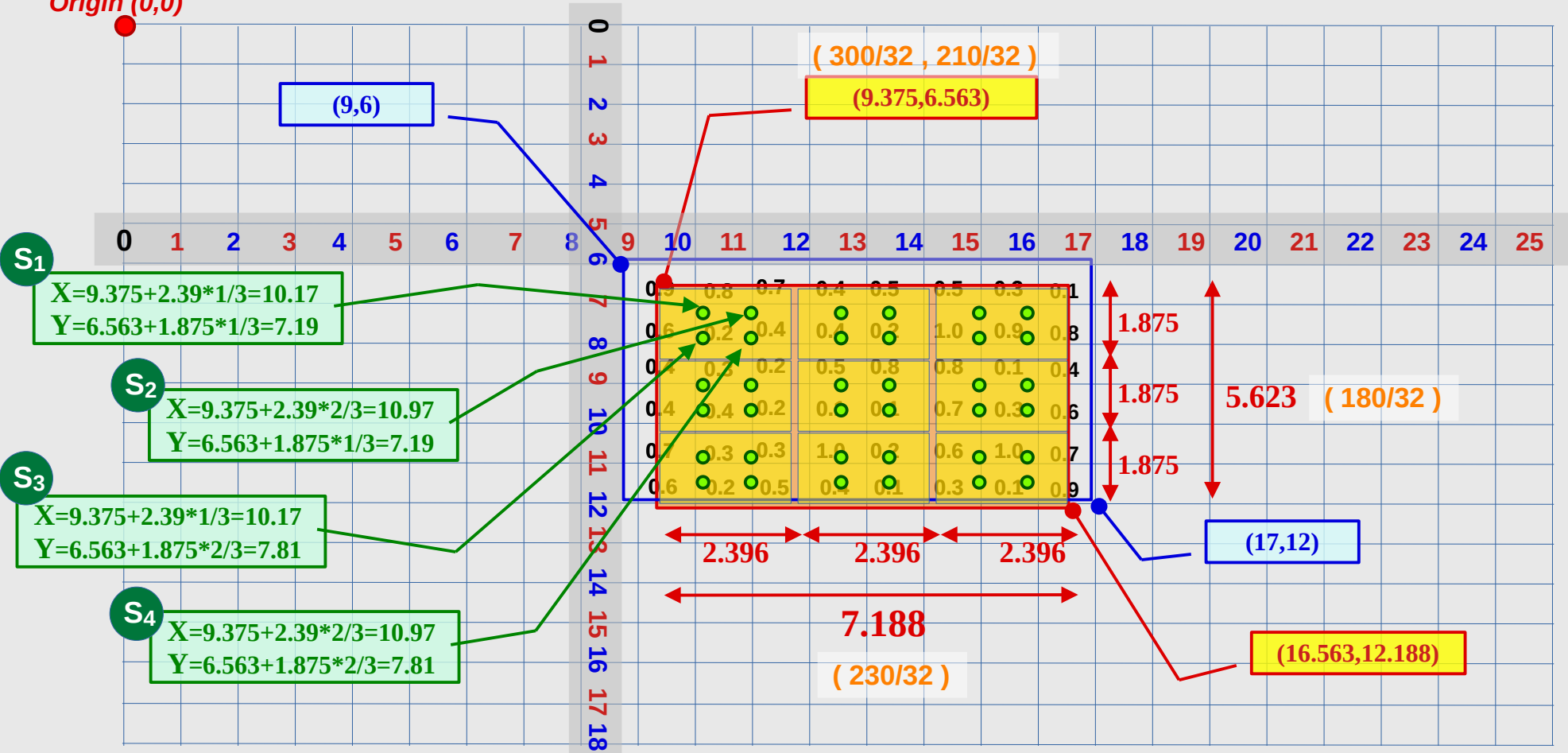
Origin (0,0)



Select 3x3 Sampling Points (Calculate Corresponding Values then apply Max Pooling)

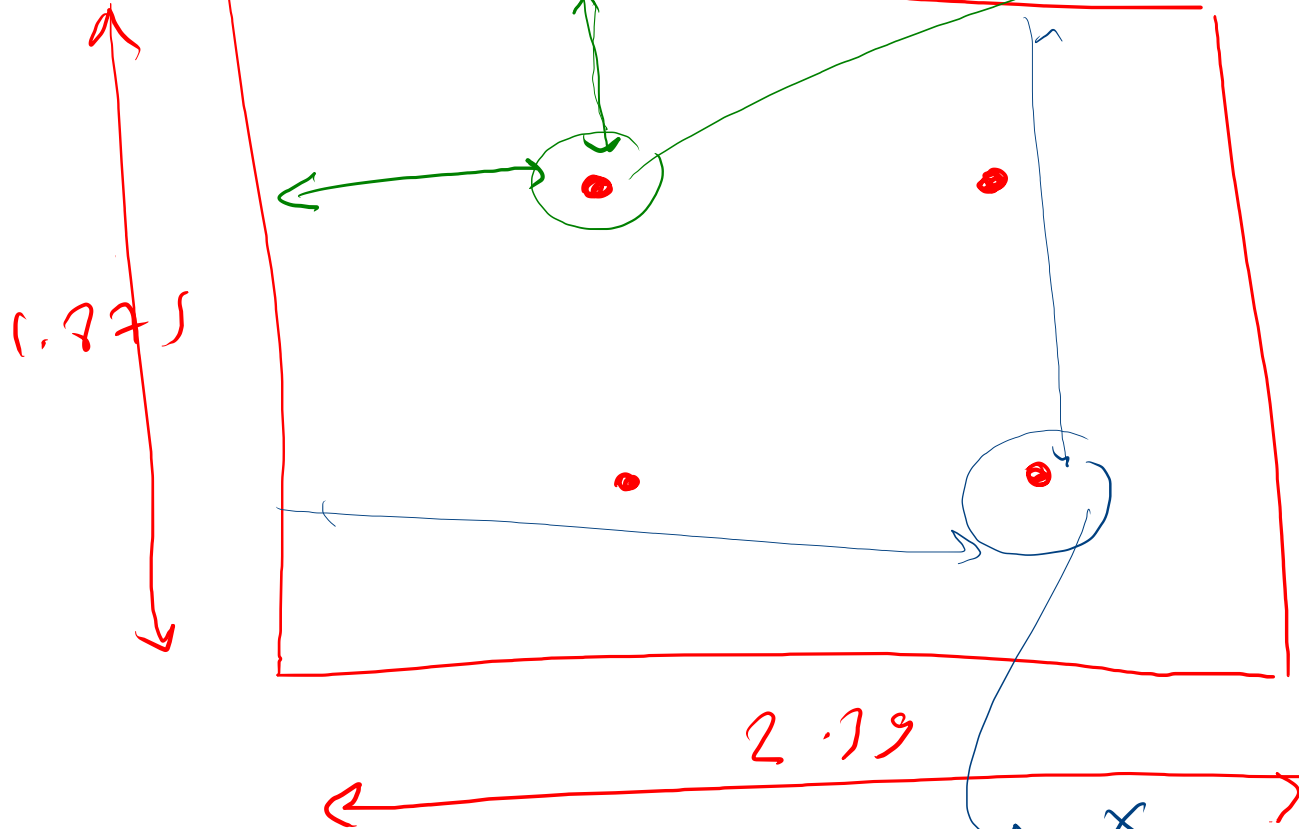
# Sampling Points Coordinates

Origin (0,0)



Feature Map Corresponding to Object at  $x,y=(300,210)$   $w,h=(230,180)$ , 5 Times Pooling

(9.77, 6.56)



$X =$

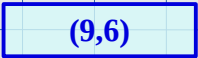
$$X = -9.77 + 2.79 x^2/7$$

$$y = 6.56 + 1.875 x^2/7$$

$$X =$$
$$y =$$

$$9.77 + 2.79 x^2/7$$
$$6.56 + 1.875 x^2/7$$

## Sampling Points Coordinates (Focus on first cell)



(9.375, 6.563)



$S_1$

$$\begin{aligned} X &= 9.375 + 2.39 * 1/3 = 10.17 \\ Y &= 6.563 + 1.875 * 1/3 = 7.19 \end{aligned}$$

**S<sub>2</sub>**

$$X = 9.375 + 2.39 \cdot \frac{2}{3} = 10.97$$
$$Y = 6.563 + 1.875 \cdot \frac{1}{3} = 7.19$$

**S<sub>3</sub>**

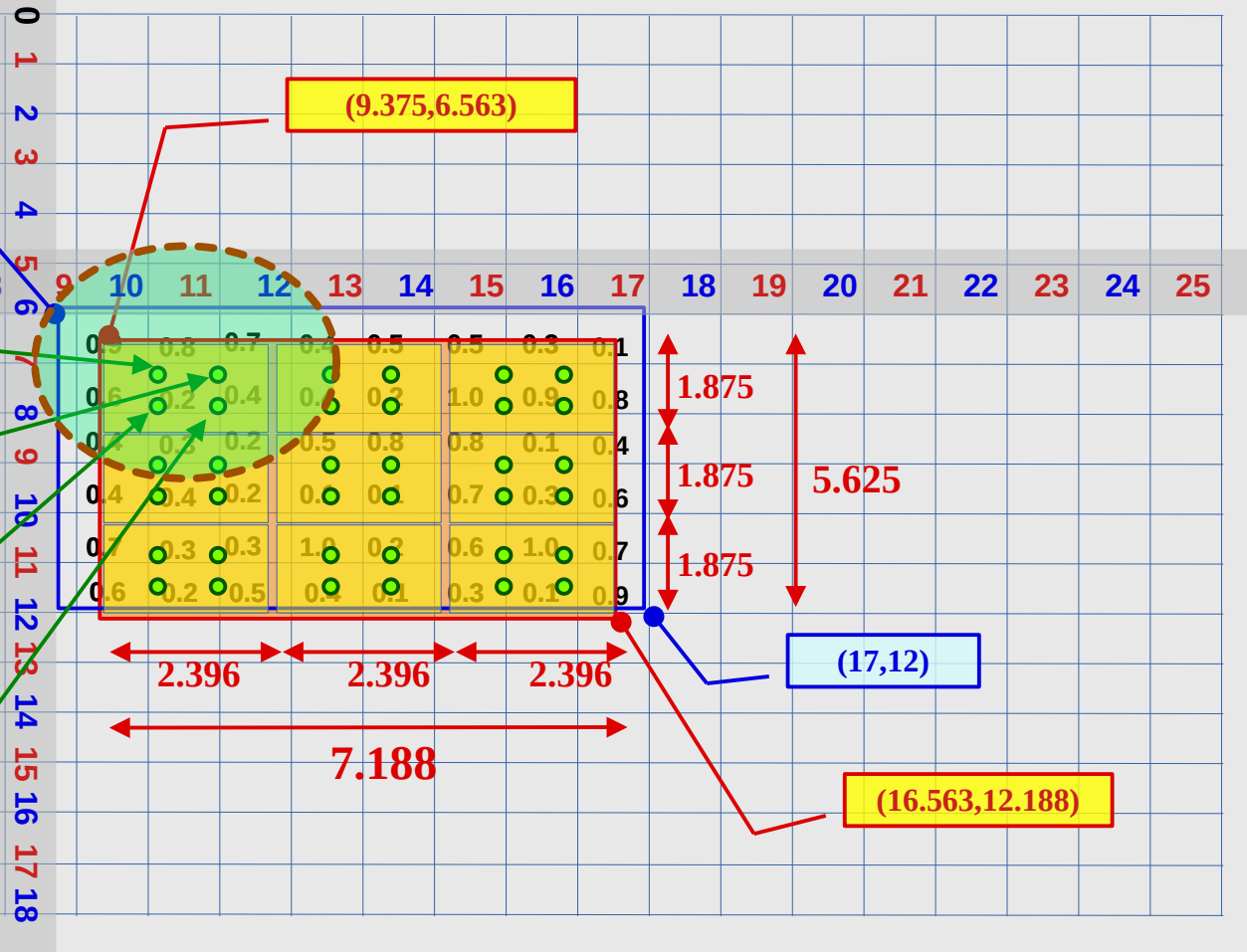
$$X = 9.375 + 2.39 \cdot \frac{1}{3} = 10.17$$
$$Y = 6.563 + 1.875 \cdot \frac{2}{3} = 7.81$$

**S<sub>4</sub>**

$$X = 9.375 + 2.39 \cdot \frac{2}{3} = 10.97$$
$$Y = 6.563 + 1.875 \cdot \frac{2}{3} = 7.81$$

(17,12)

**(16.563,12.188)**





# Linear Interpolation Example

33

Value=20

Value=70

x=3

x=4

x=5

x=6

x=7

x=8

x=9

3/6 = 50%

3/6 = 50%

Value=  $70 \cdot 3/6 + 20 \cdot 3/6 = 45$

4/6 = 66.6%

2/6 = 33.3%

Value=  $70 \cdot 4/6 + 20 \cdot 2/6 = 53.2$

1/6 = 16.6%

5/6 = 83.3%

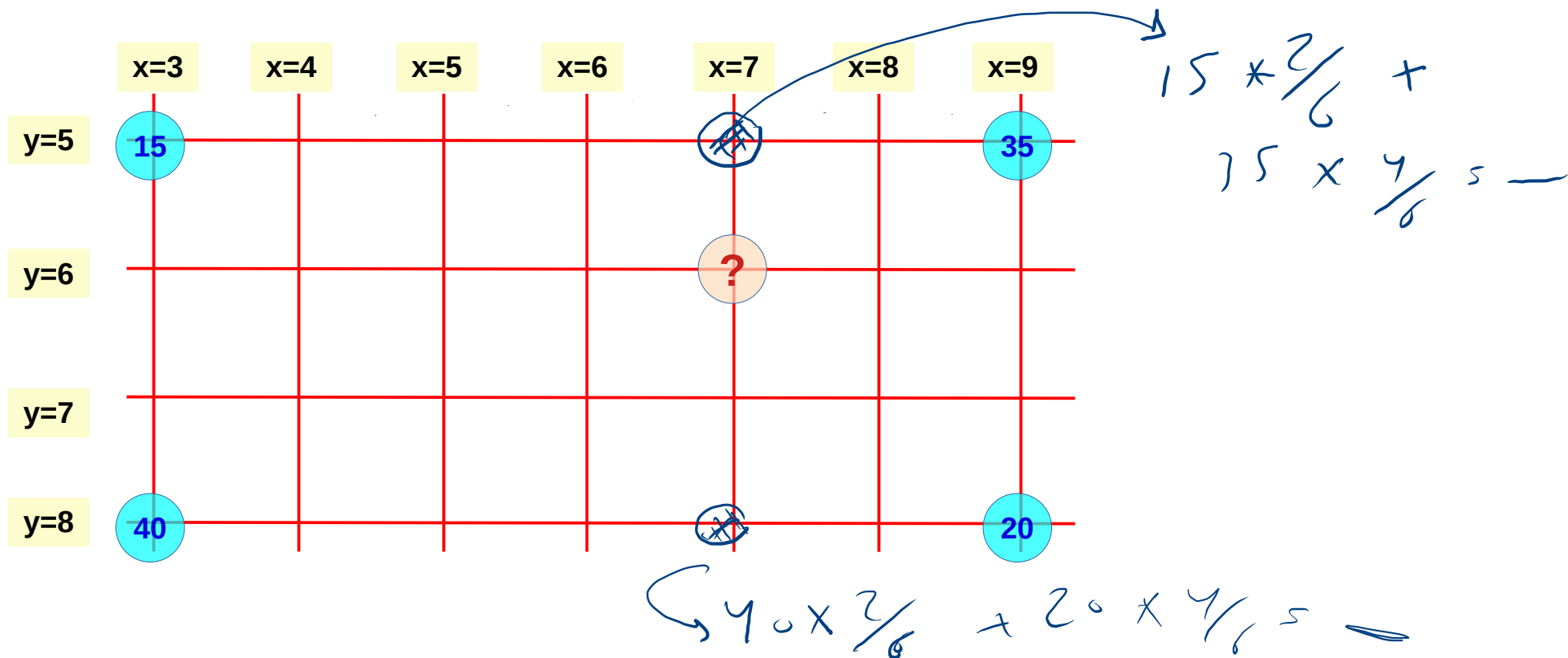
Value=  $70 \cdot 1/6 + 20 \cdot 5/6 = 28.3$

6/6 = 100%

Value=  $70 \cdot 6/6 + 20 \cdot 0/6 = 70$

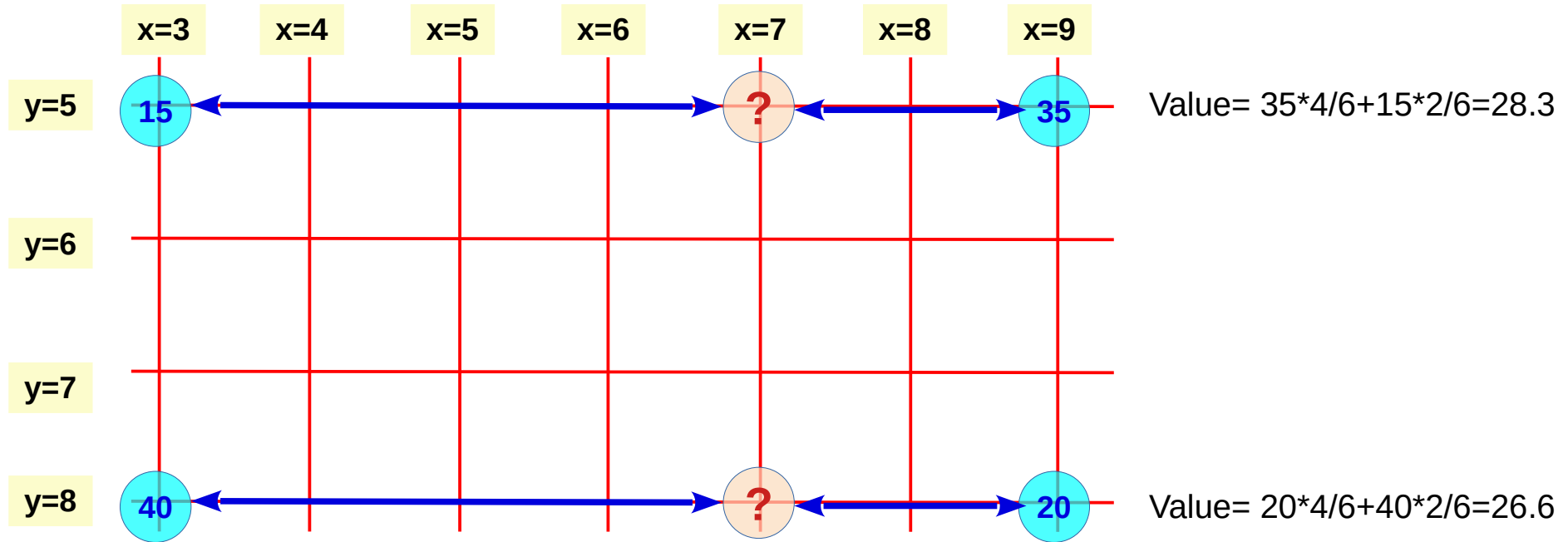
# Bi-Linear Interpolation

34



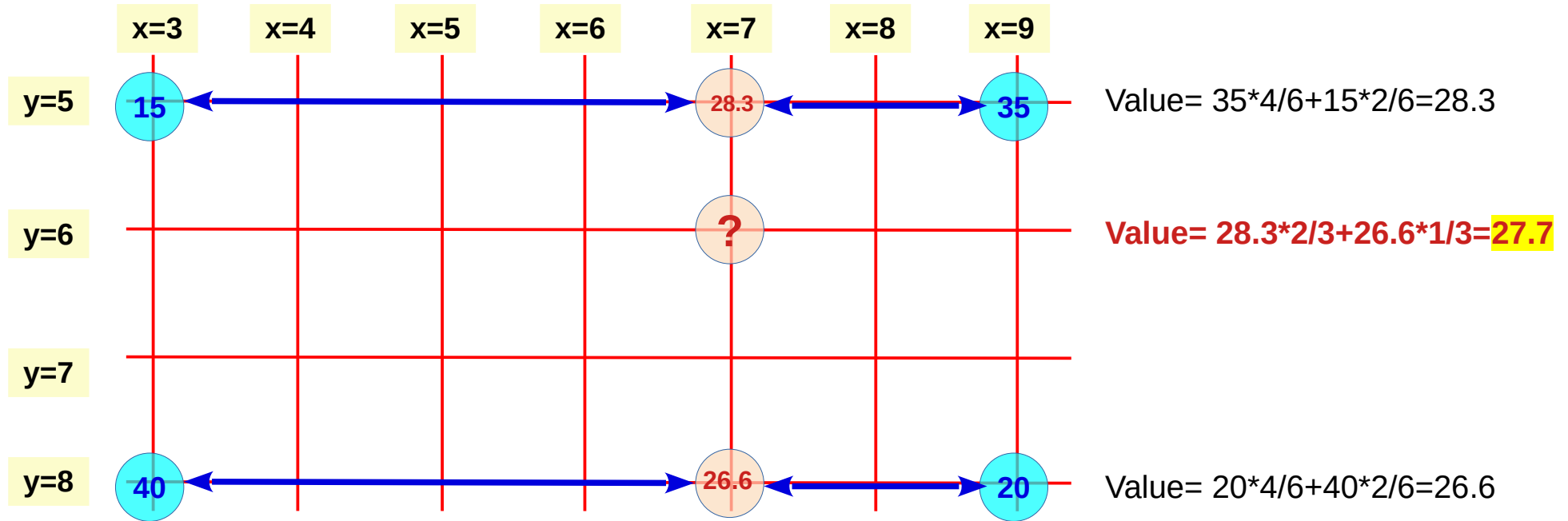
# Linear Interpolation (in X axis)

35



# Linear Interpolation (in Y - Axis)

36



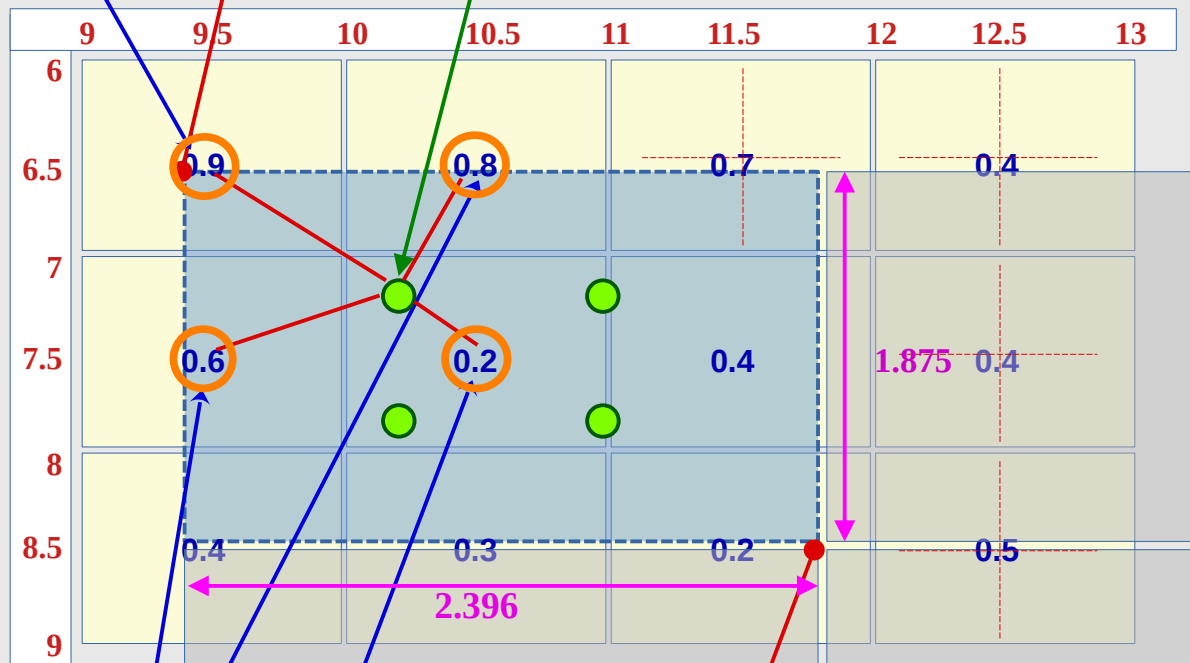
$$FM(9.5,6.5)=0.9$$

$$X,Y=(9.375,6.563)$$

$$X=9.375+2.39*1/3=10.17$$

$$Y=6.563+1.875*1/3=7.19$$

**S<sub>1</sub>**



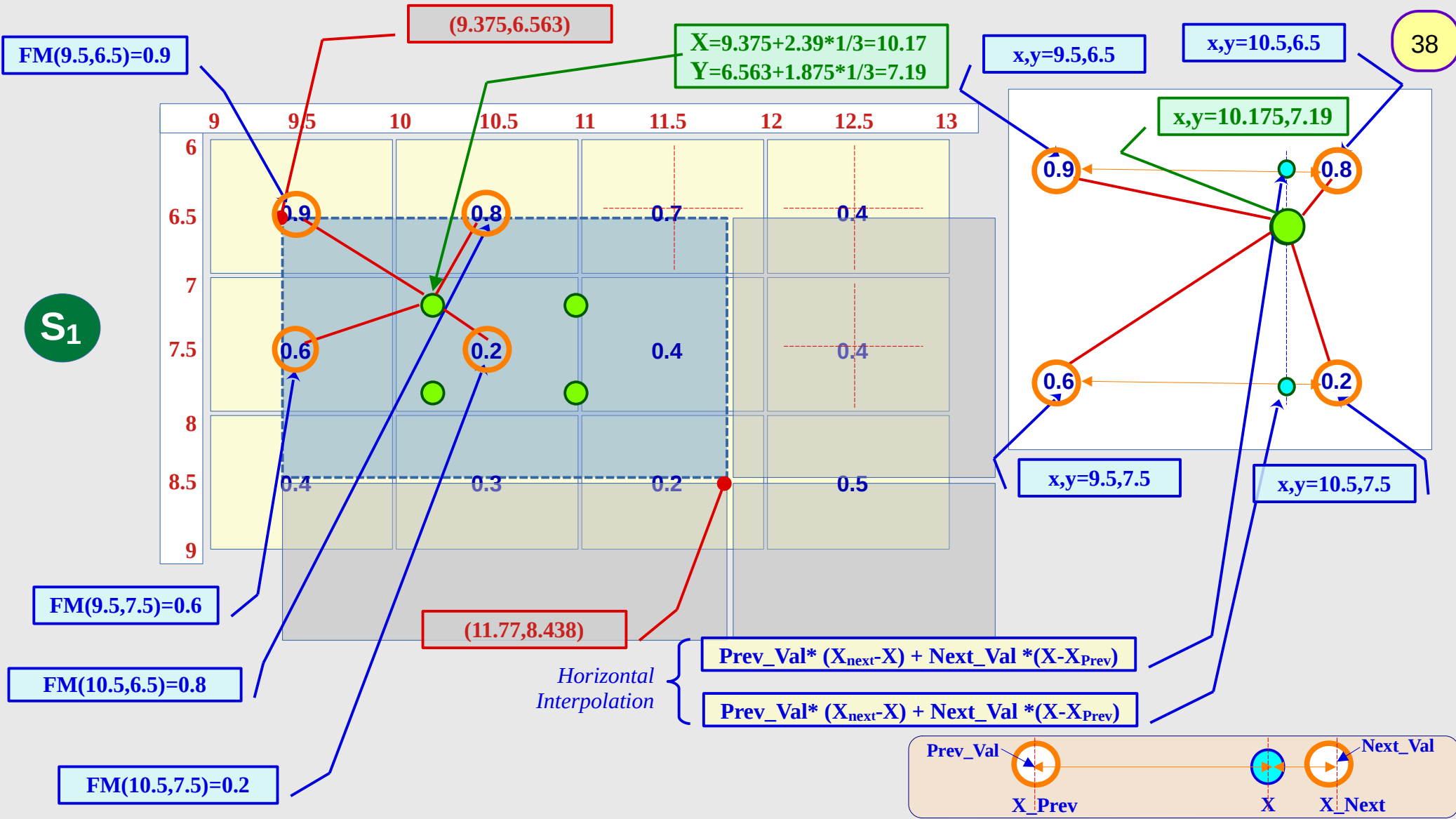
$$FM(9.5,7.5)=0.6$$

$$FM(10.5,6.5)=0.8$$

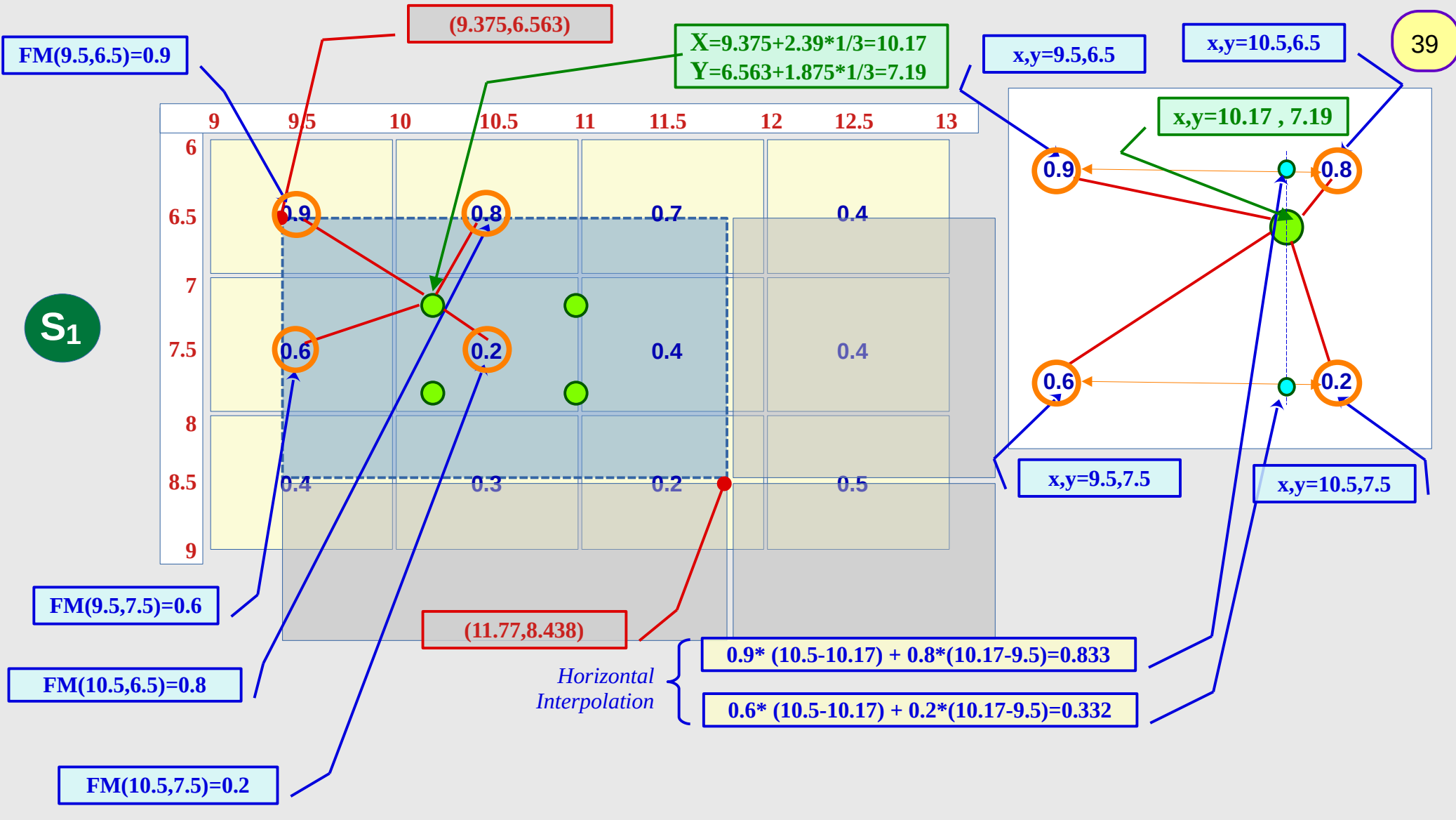
$$FM(10.5,7.5)=0.2$$

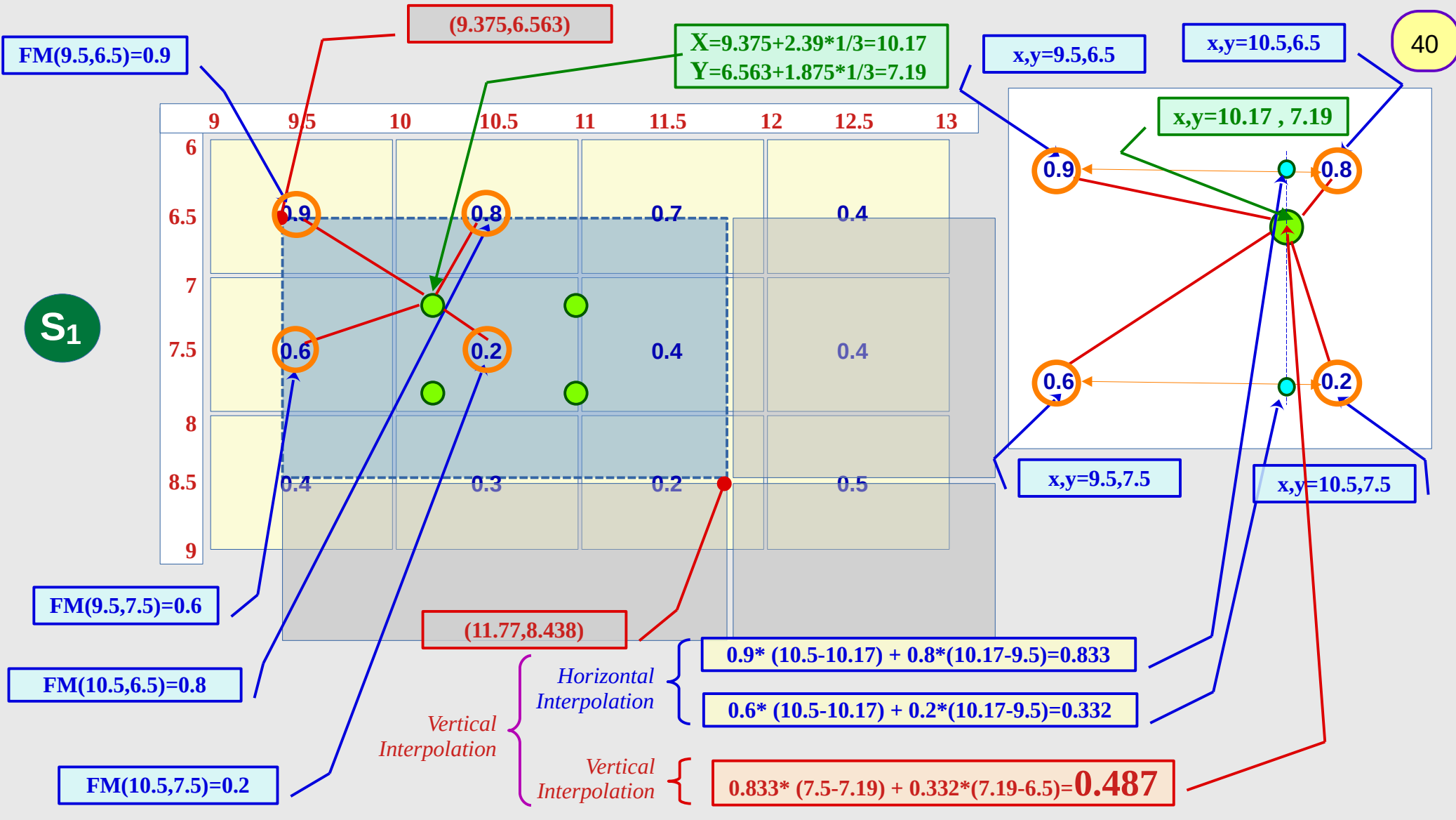
$$X,Y=(11.77,8.438)$$











$$\text{FM}(10.5, 6.5) = 0.8$$

(9.375, 6.563)

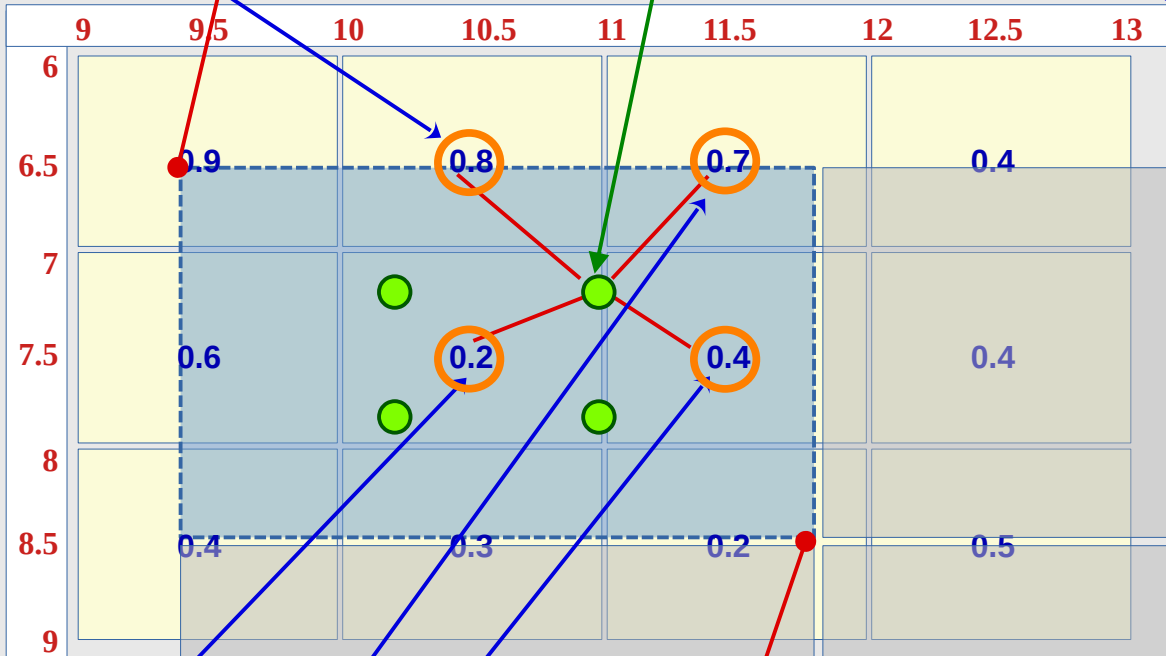
$$X = 9.375 + 2.39 \cdot \frac{2}{3} = 10.97$$
$$Y = 6.563 + 1.875 \cdot \frac{1}{3} = 7.19$$

**x,y=10.5,6.5**

**x,y=11.5,6.5**

41

**x,y=10.97 , 7.19**



**FM(10.5,7.5)=0.2**

$$\text{FM}(11.5,6.5)=0.7$$

FM(11.5,7.5)=04

(11.77, 8.438)

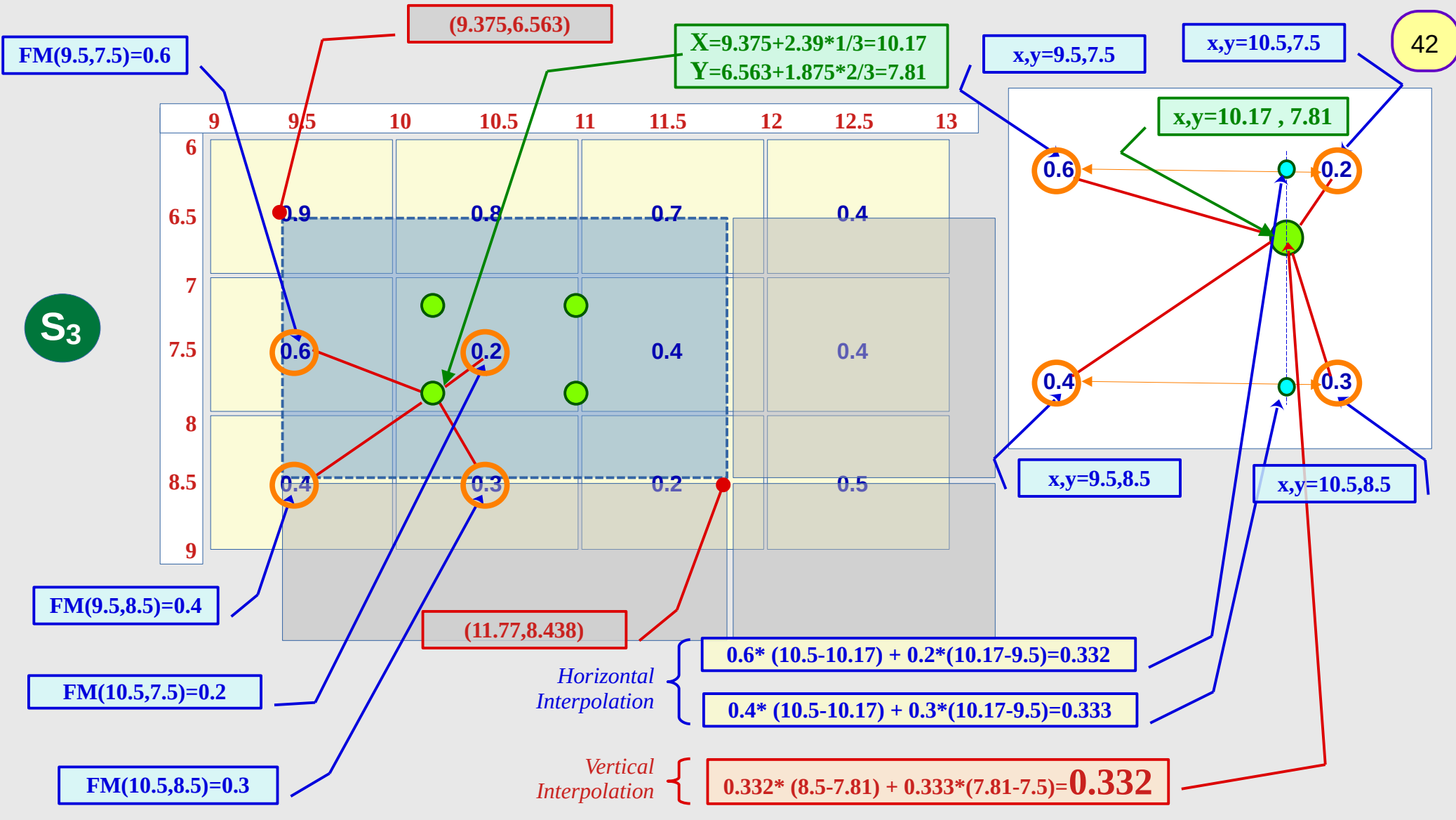
## Horizontal Interpolation

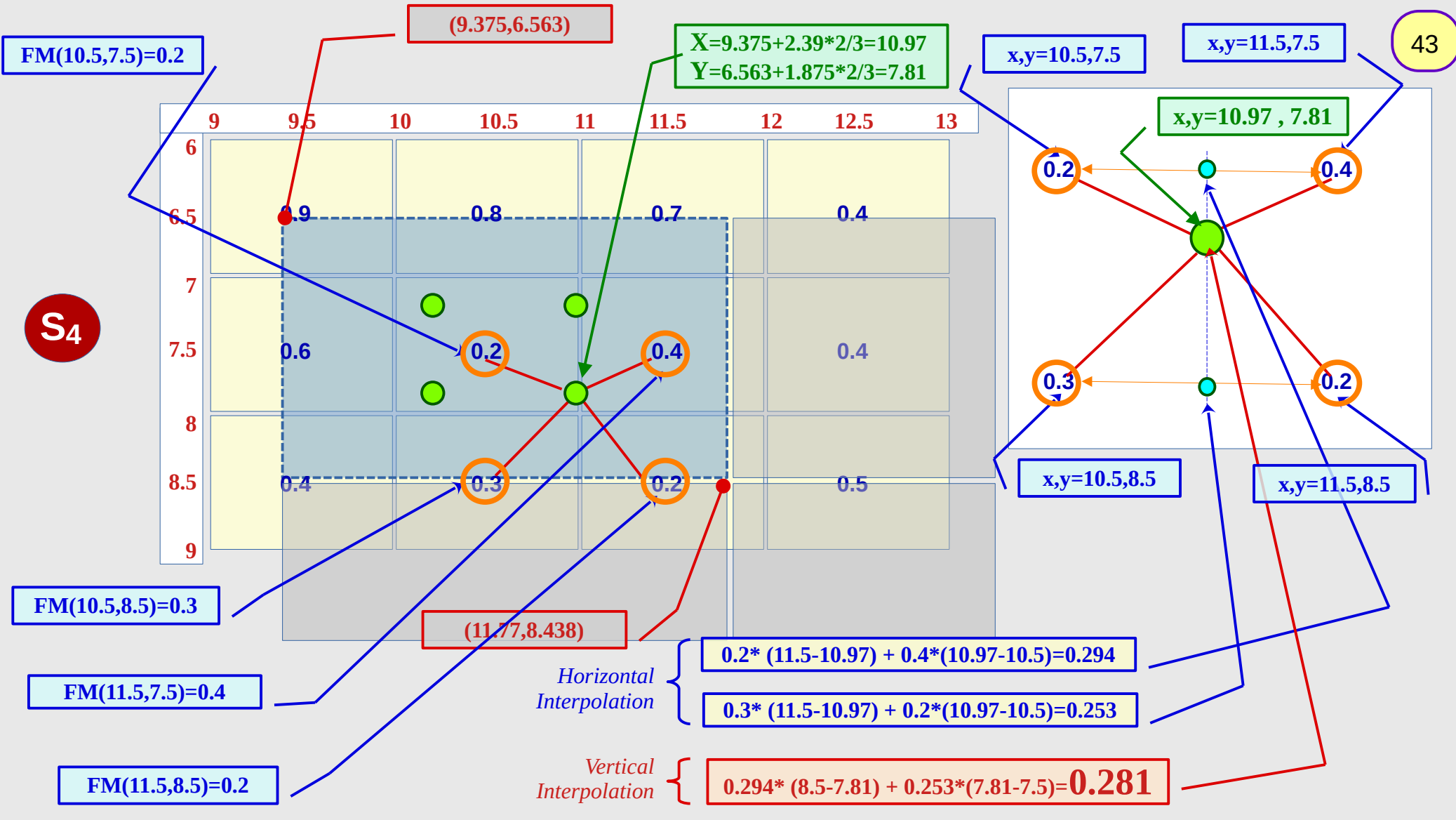
$$0.8 * (11.5 - 10.97) + 0.7 * (10.97 - 10.5) = 0.753$$

$$0.2 * (11.5 - 10.97) + 0.4 * (10.97 - 10.5) = 0.294$$

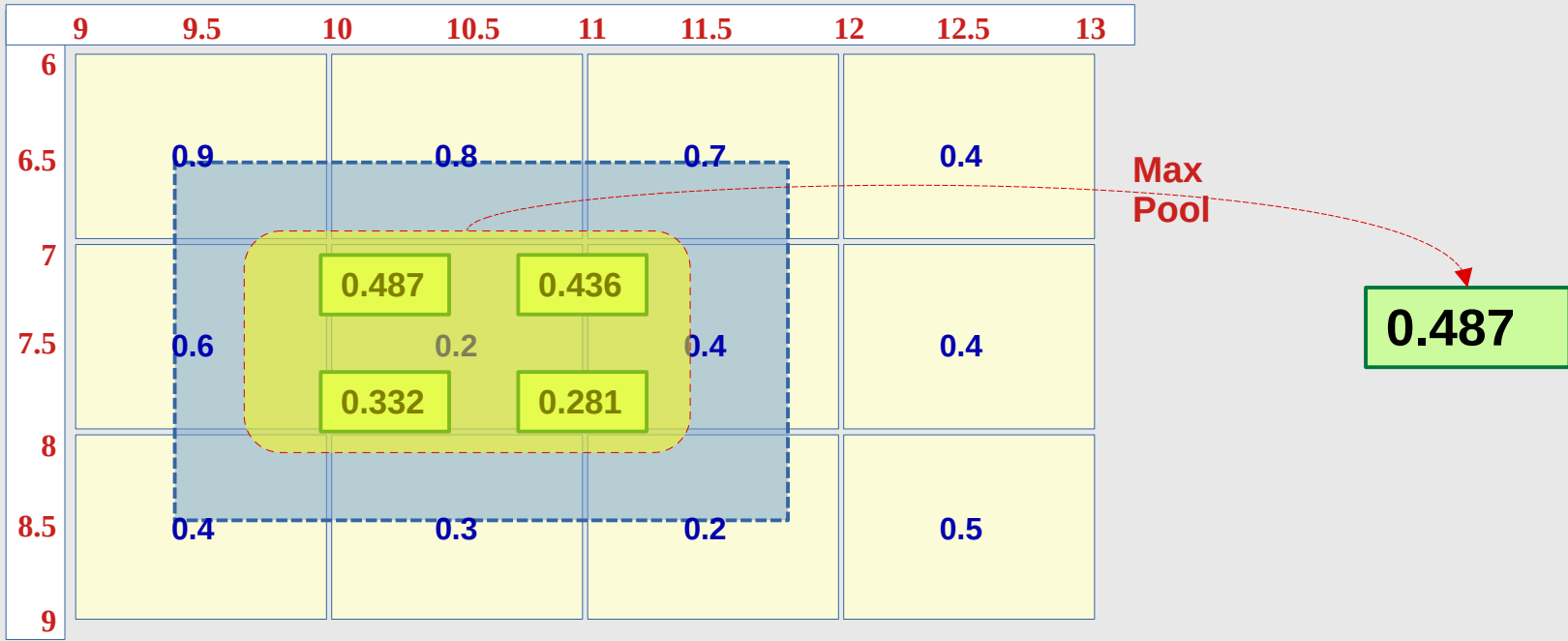
## Vertical Interpolation

$$0.753* (7.5-7.19) + 0.294*(7.19-6.5)=0.436$$

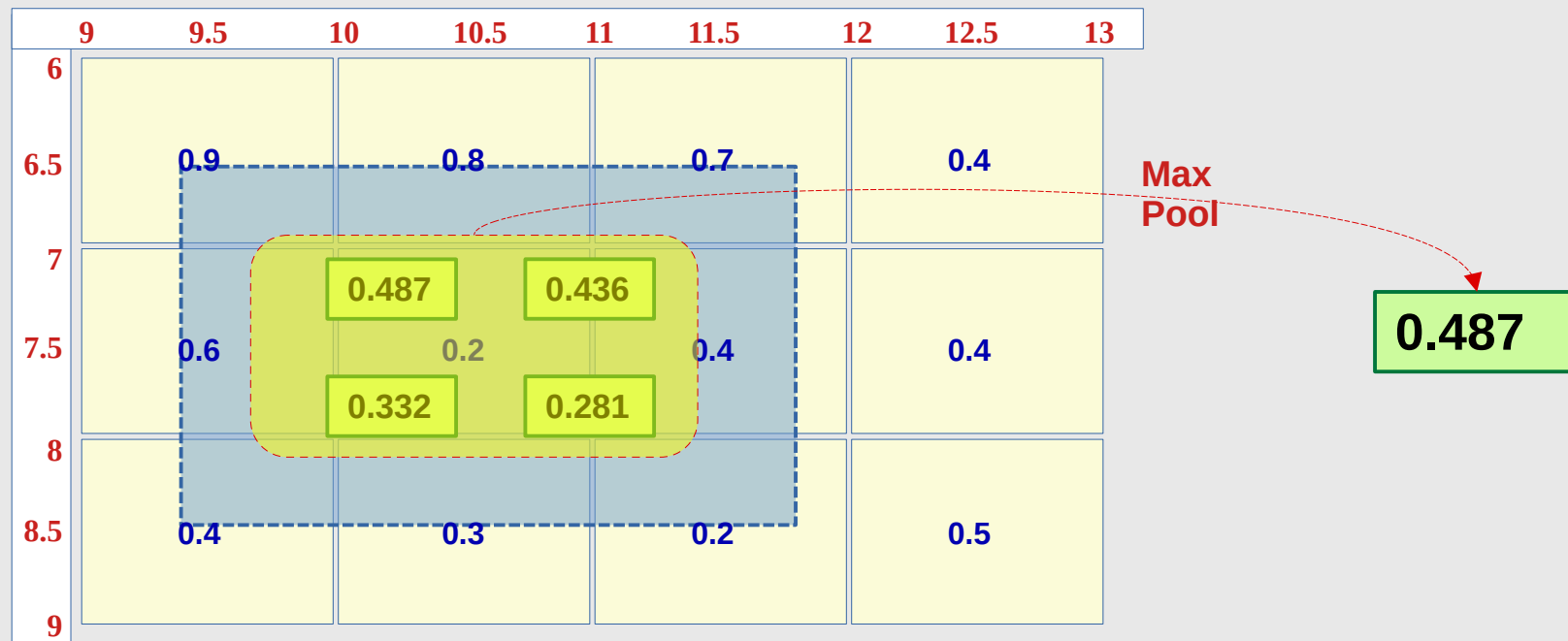




## Apply Max-Pooling on the Calculated Sampling Points



## Apply Max-Pooling on the Calculated Sampling Points



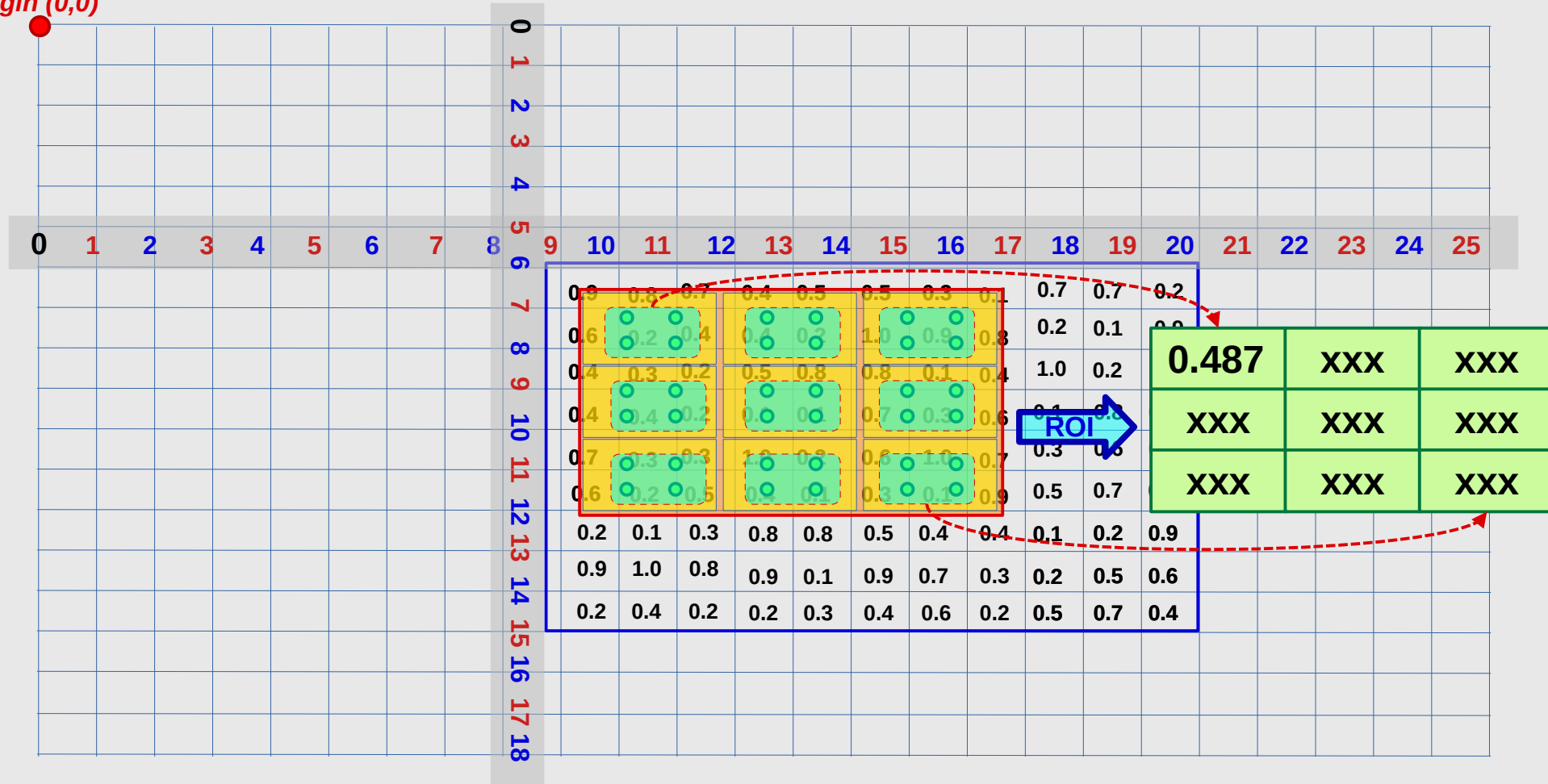
*What if we use 3x3 sampling points (instead of 2x2 points)*

*Repeat for All other Cells Within the Region Of Interest (ROI) to obtain 3x3 values*



# 3x3 ROI Align for selected Object

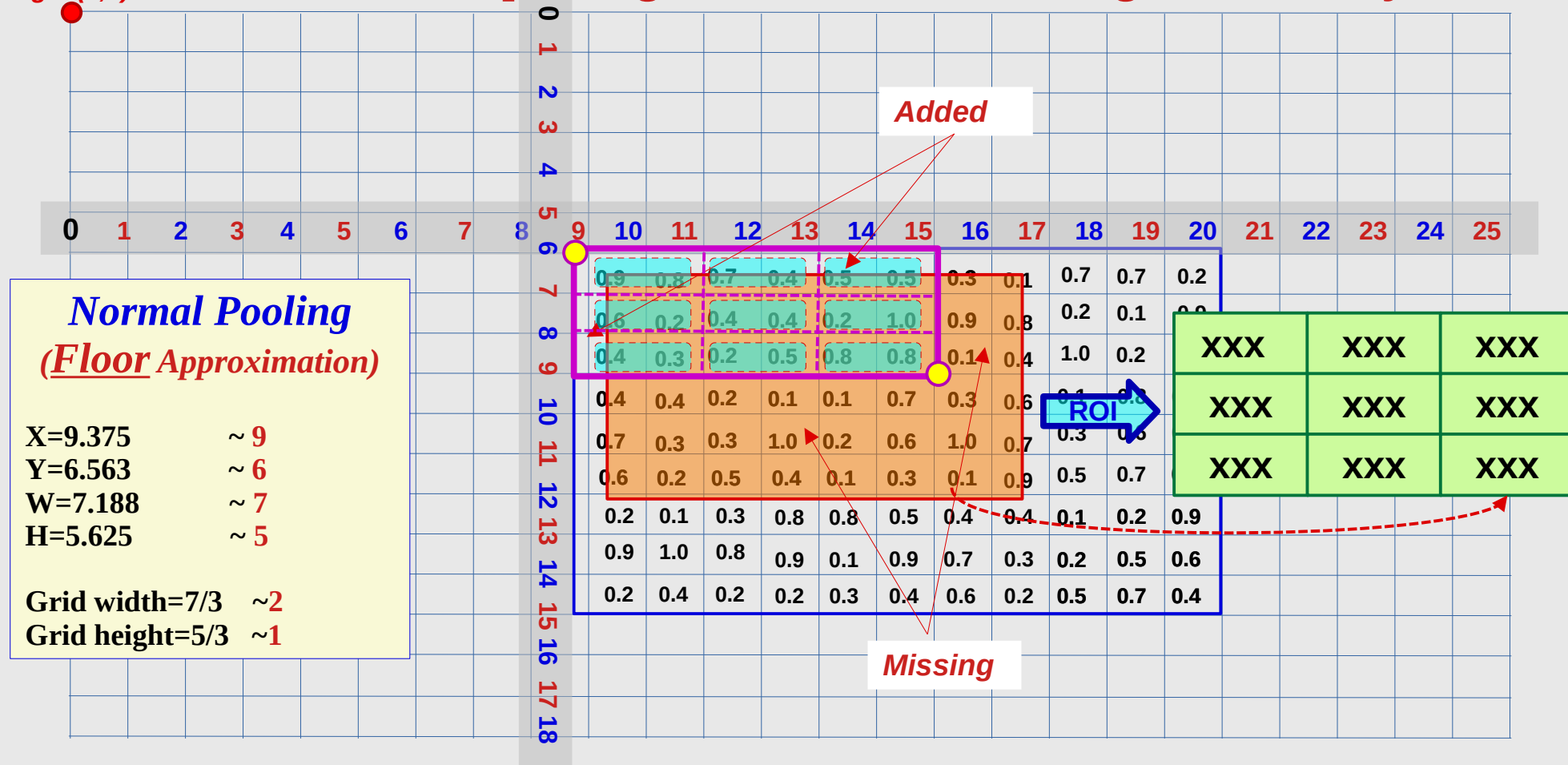
Origin (0,0)



Remember:

Origin (0,0)

## Corresponding Traditional ROI Pooling for same Object



## Remember:

**Origin (0,0)**

## Corresponding Traditional ROI Pooling for same Object

## Normal Pooling (Round Approximation)

**X=9.375**      **~ 9**

**Y=6.563**      **~ 7**

**W=7.188**       $\approx$  **7**

**H=5.625**      **~ 6**

**Grid width=7/3    ~2**

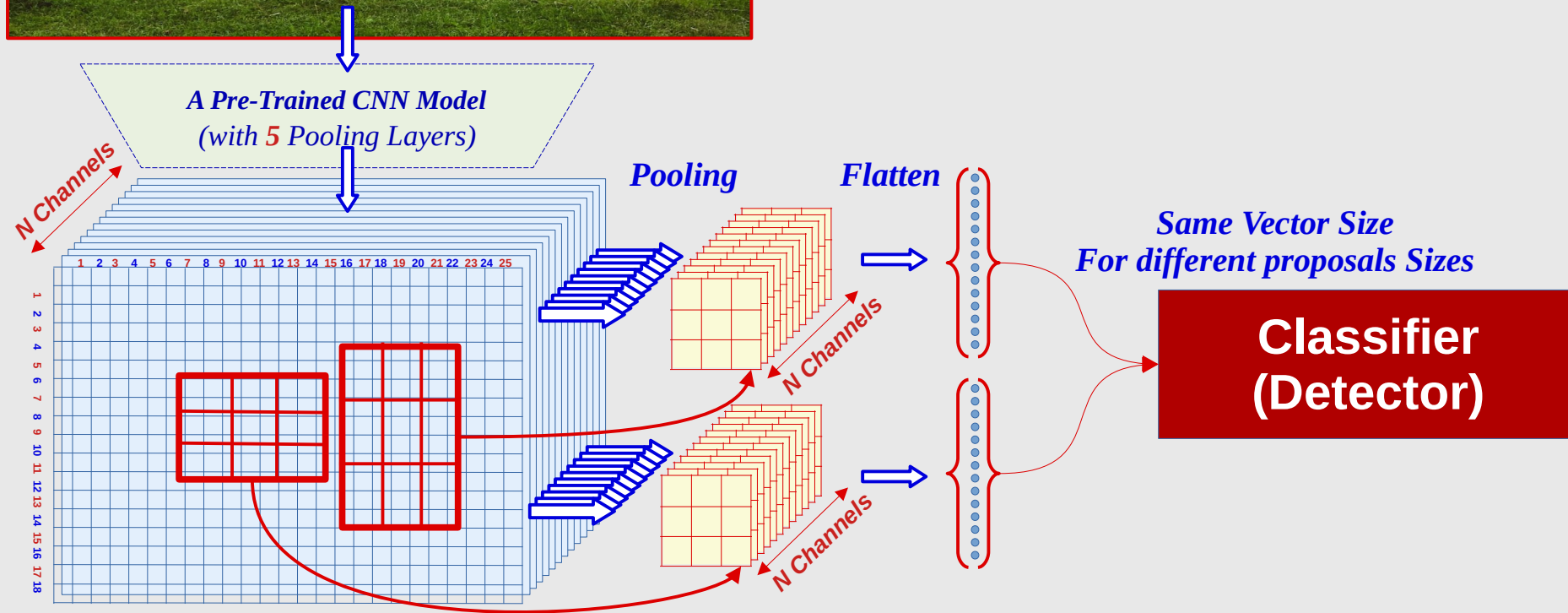
**Grid height=6/3 ~2**

**Missing**

ROI

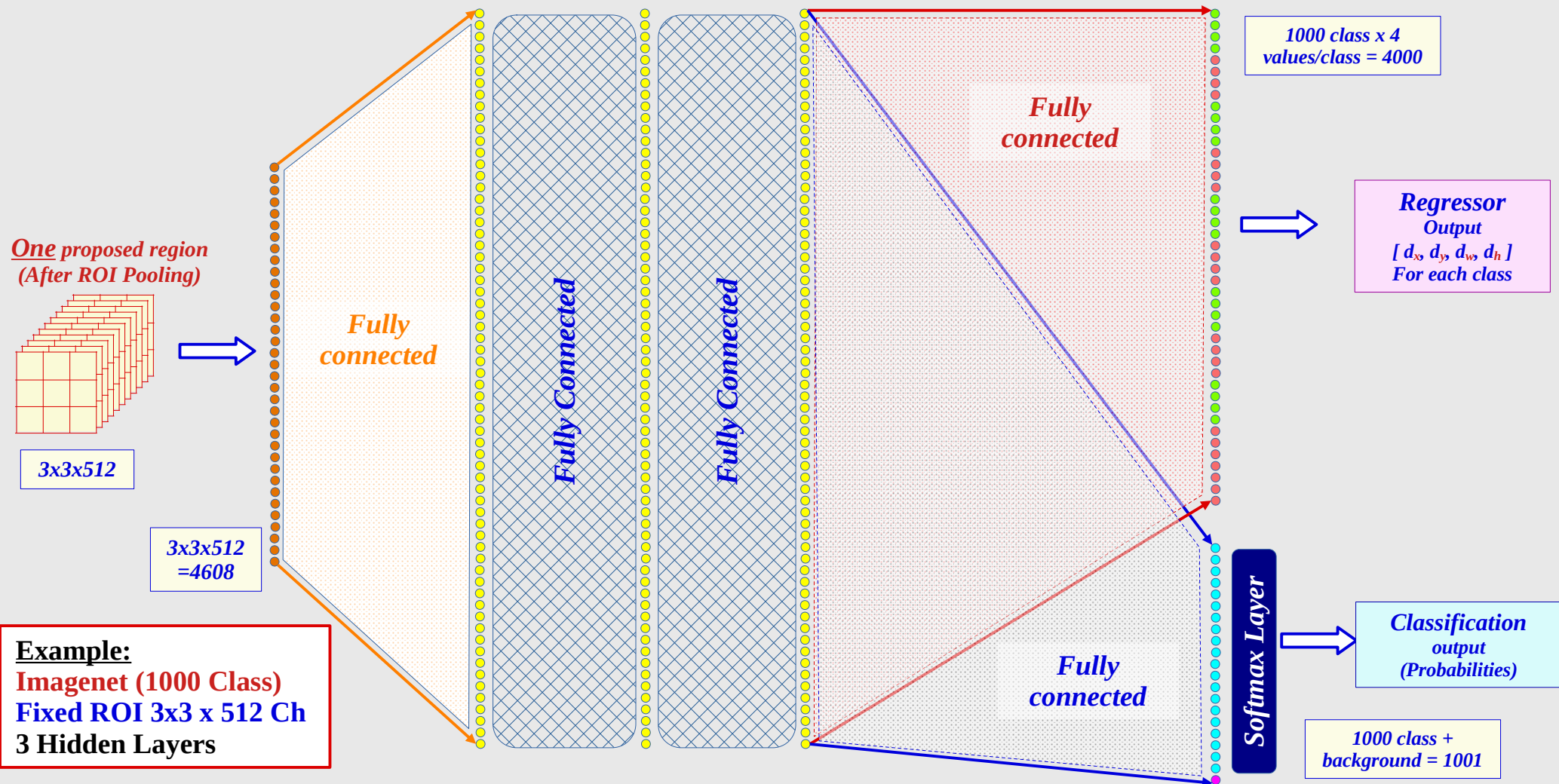
**Added**

*Max – Pooling is applied on Channel Basis  
(repeat pooling for Each Channel)*



# Architecture of Detector Network (for Fast RCNN, Faster RCNN)

50



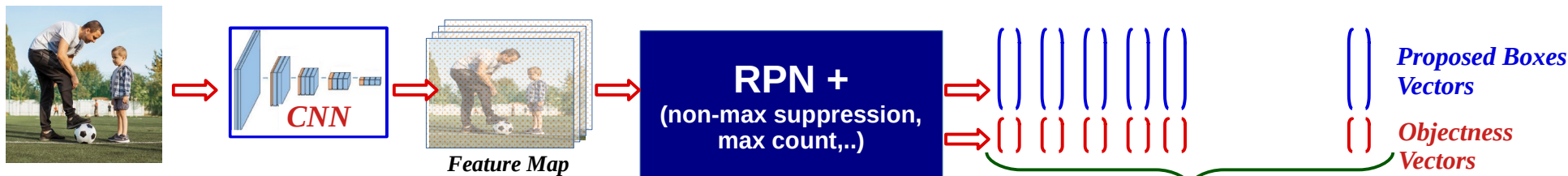
# Training of Faster RCNN Detector



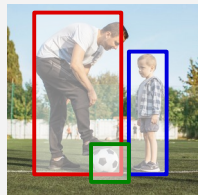
**Run RPN on a Randomly-selected training Image**

*Filter overlapped Proposed Regions using Non-Max-Suppression*

*Limit the number of proposed regions to a predefined number*



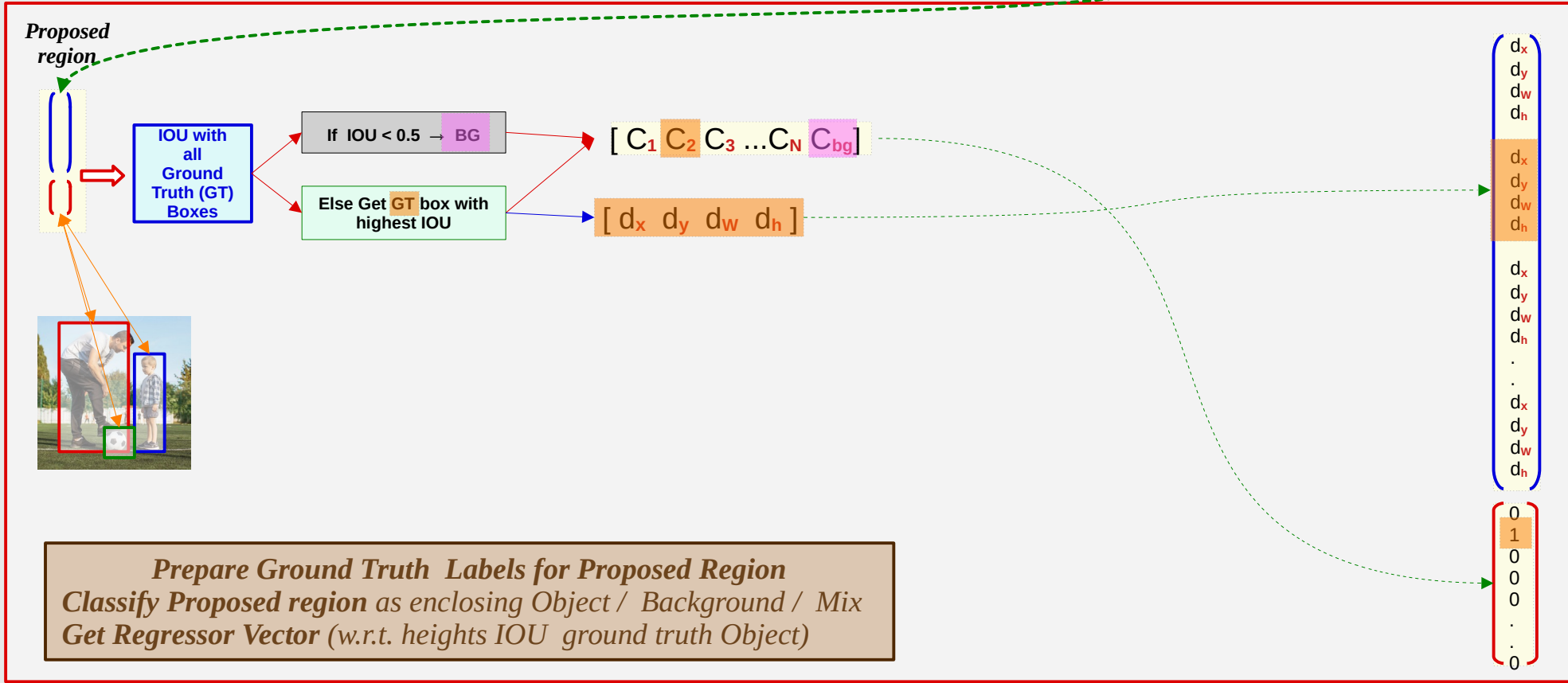
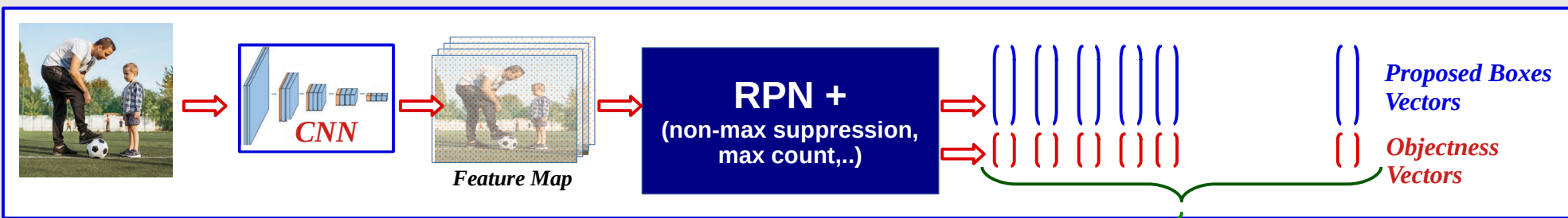
*Proposed region*

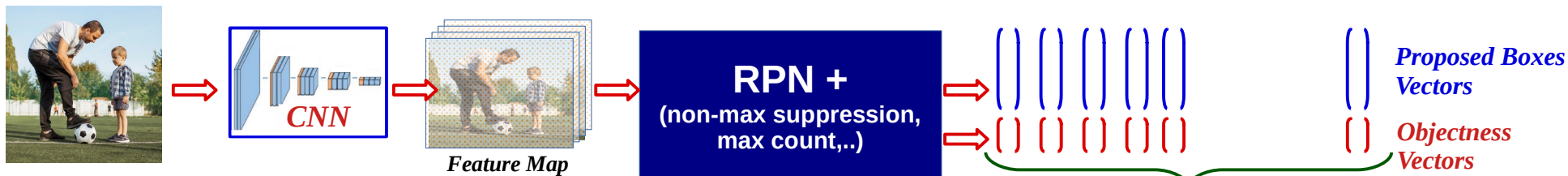


***Pick Proposed Region Vectors (one by one)***

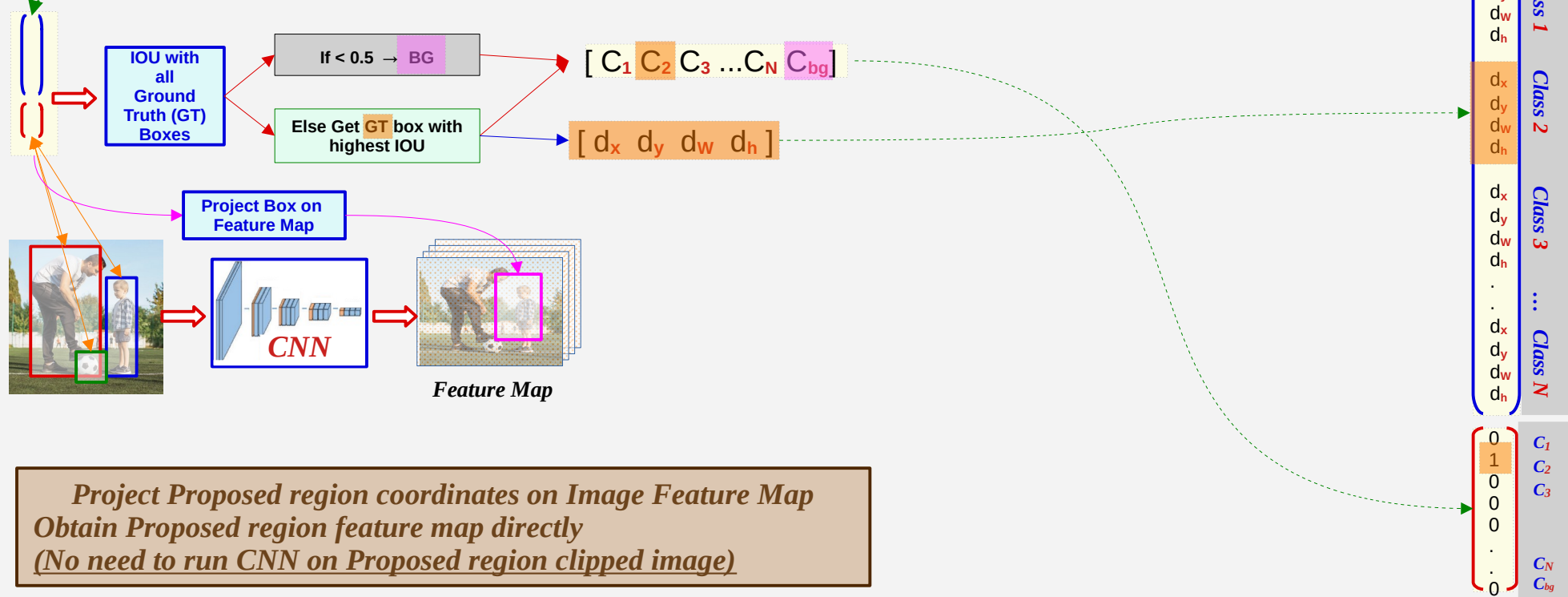
*Supply Proposed region Vectors ( **Regressor and Objectness**) in conjunction with **Original Image** to Train Fast RCNN Detector*



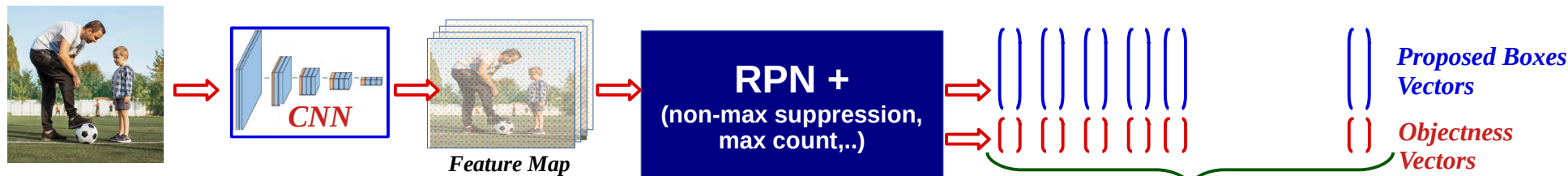




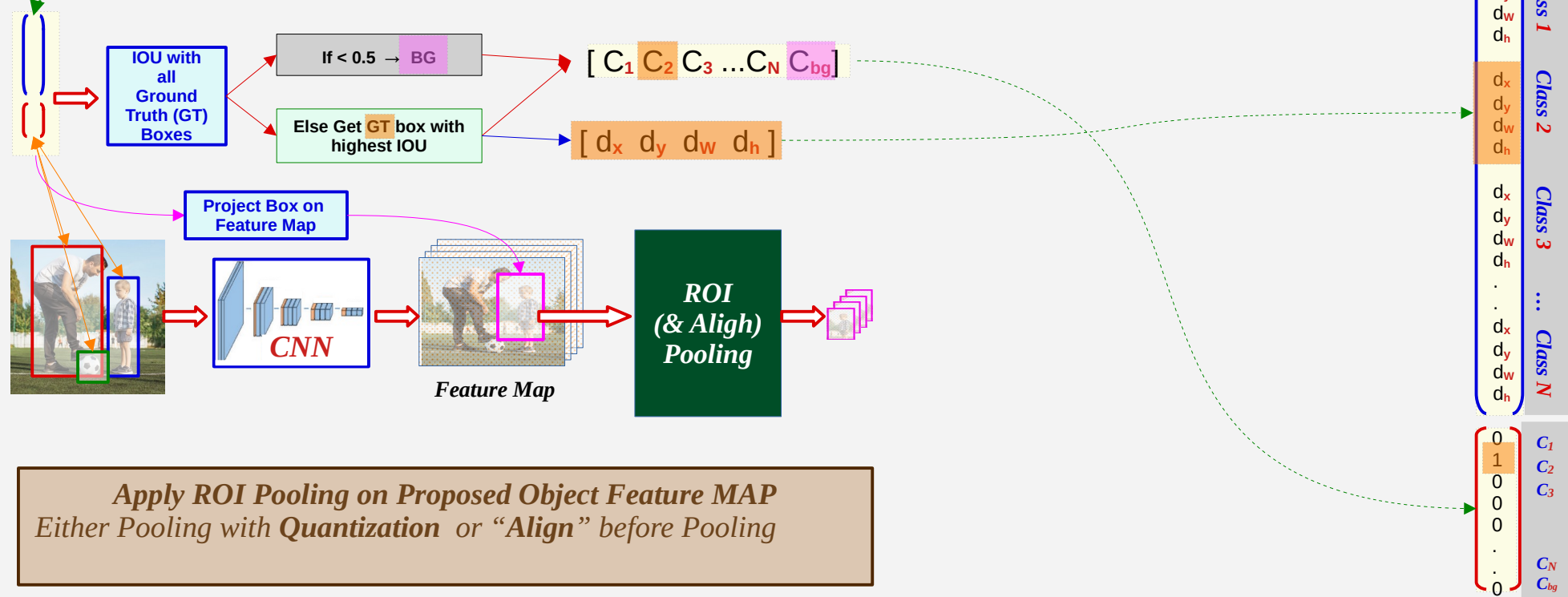
*Proposed region*



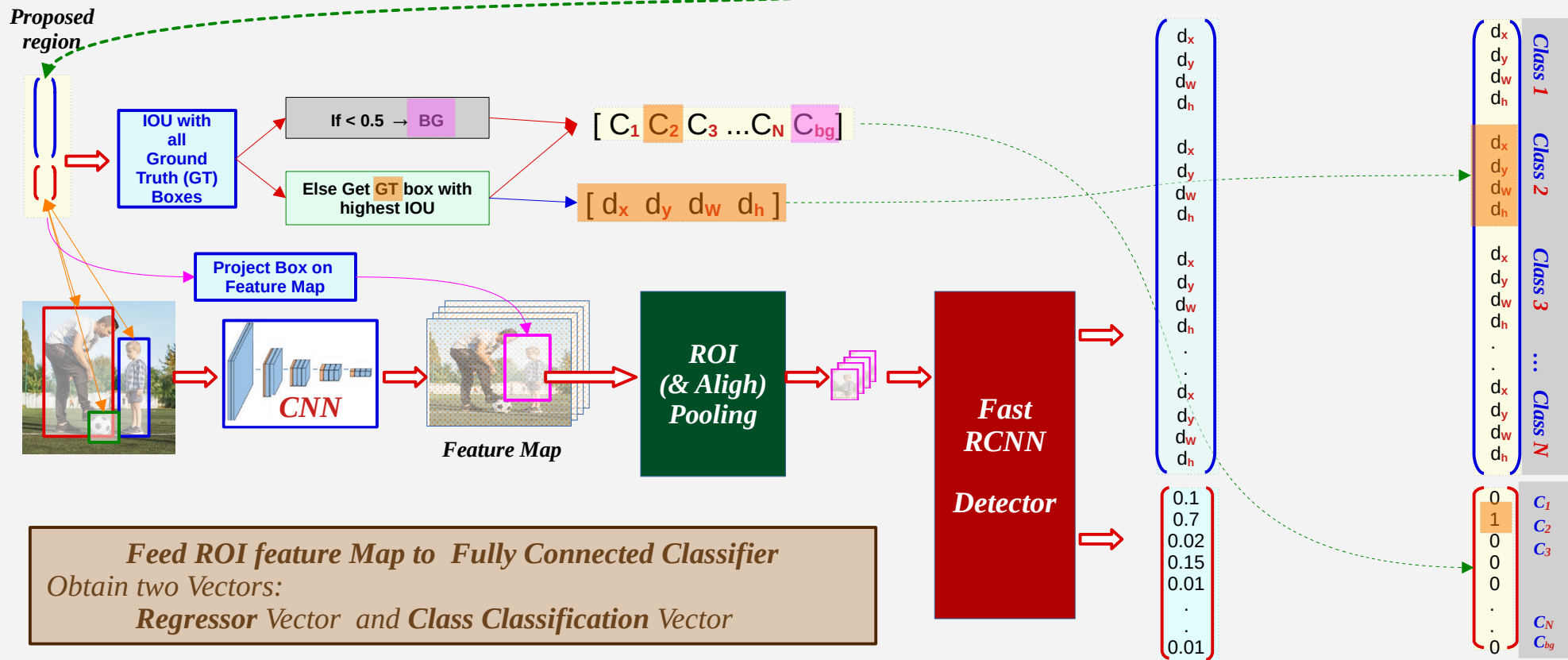
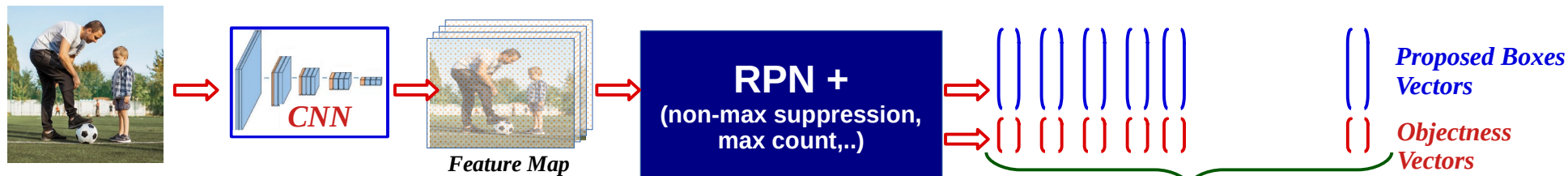
*Project Proposed region coordinates on Image Feature Map  
Obtain Proposed region feature map directly  
(No need to run CNN on Proposed region clipped image)*

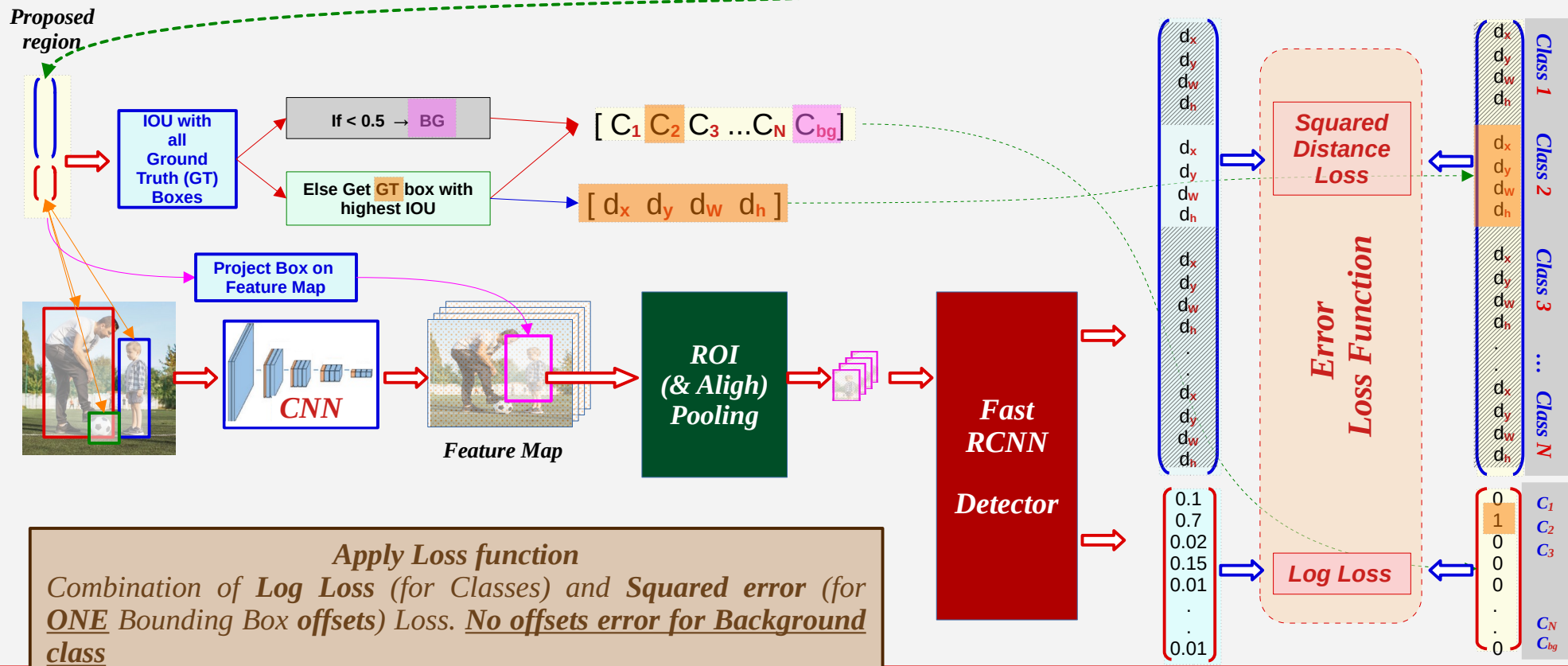
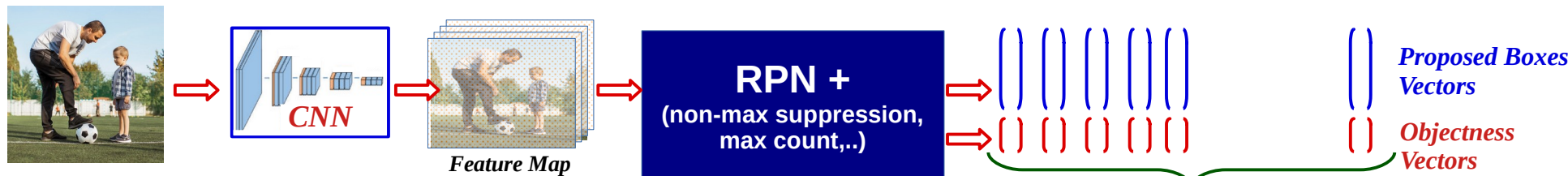


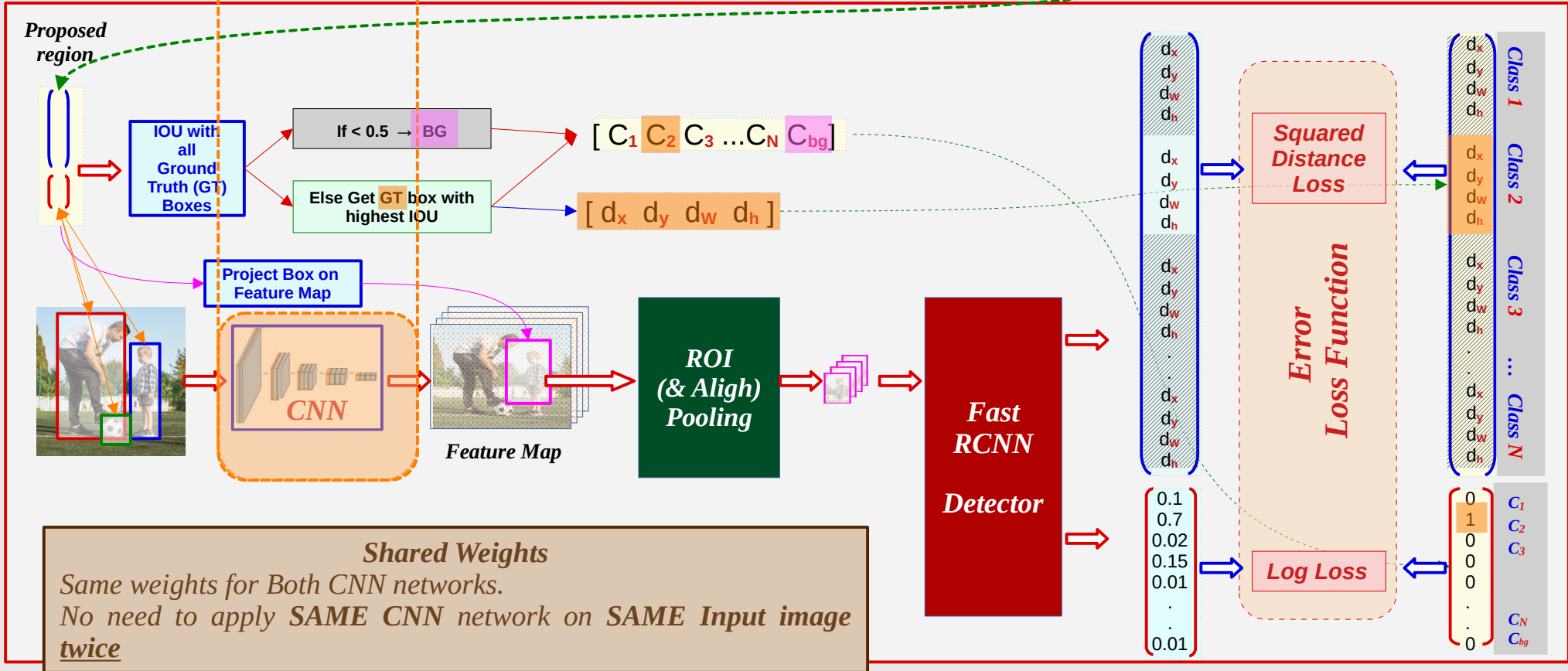
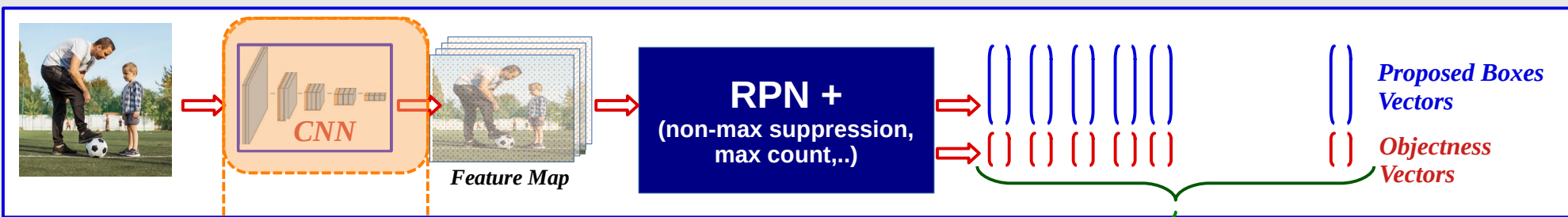
Proposed region



Apply ROI Pooling on Proposed Object Feature MAP  
Either Pooling with **Quantization** or "Align" before Pooling







**Shared Weights**  
Same weights for Both CNN networks.  
No need to apply **SAME** CNN network on **SAME** Input image twice



